

From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akkradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a small, statically-typed interpreted programming language realised through a four-stage compiler pipeline. The language provides four primitive types — Integer, Float, Boolean, and String — and supports arithmetic and equality expressions, variable assignment with type inference from first use, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` function. Types are inferred from program context rather than declared explicitly, combining static type safety with a declaration-free syntax. The pipeline consists of a hand-written lexer, a recursive-descent parser that constructs an abstract syntax tree, a static type checker that performs inference and rejects ill-typed programs before execution, and a tree-walking interpreter. Both a command-line runner and an interactive desktop interface expose each pipeline stage for inspection. The report documents the formal grammar, the type inference algorithm, the system architecture, and the automated test suite of 54 tests that validates the complete implementation.

CONTENTS

ABSTRACT	ii
1 INTRODUCTION	1
2 LANGUAGE DESIGN AND GRAMMAR	3
2.1 Types	3
2.1.1 Integer	3
2.1.2 Float	3
2.1.3 Boolean	3
2.1.4 String	3
2.1.5 Void	4
2.2 Static Typing	4
2.3 Token Specification	4
2.4 Context-Free Grammar	5
2.4.1 Program Structure	5
2.4.2 Arithmetic Expressions	5
2.4.3 Boolean Expressions	6
2.4.4 Statements	6
2.4.5 Function Definitions and Calls	6
2.5 Operator Precedence and Associativity	6
3 LEXICAL ANALYSIS AND SYMBOL TABLE	8
3.1 Token representation	8
3.2 Lexer implementation	9
3.3 Symbol table and semantic structure	11

3.4	Role of the symbol table in the pipeline	12
4	RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE	14
4.1	Abstract syntax tree representation	14
4.1.1	Expressions	15
4.1.2	Statements	15
4.2	Parser implementation	16
4.2.1	Statement parsing	16
4.2.2	Expression parsing and precedence	17
4.2.3	Helper functions and lookahead	18
4.3	AST printing	18
4.4	Parser and type checker boundary	19
5	TYPE INFERENCE ALGORITHM	20
5.1	Overview	20
5.2	Type Environment	20
5.3	Type Inference Procedure	20
5.4	Inference Rules	22
5.4.1	Literals	22
5.4.2	No Operator Overloading	22
5.4.3	Arithmetic Expressions	23
5.4.4	Boolean Expressions	23
5.4.5	Assignment Statement	23
5.4.6	If-Then-Else	24
5.4.7	While-Loop	24
5.4.8	Function Definition and Call	24
5.5	Type Error Reporting	24
5.6	Example Type Inference Traces	25

6	SYSTEM ARCHITECTURE AND IMPLEMENTATION	26
6.1	Compiler pipeline	26
6.1.1	End-to-end pipeline walkthrough	27
6.1.2	Entry points and shared pipeline	28
6.2	Project structure	28
6.3	Technology stack	29
6.4	Type checker	29
6.4.1	Scope of the implemented type system	29
6.5	Interpreter	30
6.5.1	Scope of the implemented runtime	30
6.6	Graphical user interface	30
6.6.1	Screenshots	31
6.7	Command-line interface	32
7	TESTING AND VERIFICATION	34
7.1	Manual Testing	34
7.1.1	Lexer and Symbol Table	34
7.1.2	Parser and AST	35
7.1.3	Arithmetic Expressions	36
7.1.4	Boolean Expressions	36
7.1.5	Assignment Statements	36
7.1.6	If-Then-Else	36
7.1.7	While-Loop	36
7.1.8	Functions	36
7.1.9	Print	37
7.1.10	Unary Minus	37
7.2	Type Error Detection	37
7.3	Automated Test Suite	37

7.4 Error Handling	39
8 CONCLUSION	40
REFERENCES	41
APPENDIX A: SOURCE CODE	42
APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS	42

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer’s intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a small statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` statement. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser and abstract syntax tree, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the four pipeline stages — tokens, AST, inferred type table, and execution output — for inspection. An automated regression suite of 54 tests verifies correctness across all stages and language features.

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and the semantic rules used by the checker. Chapter 6 presents the system architecture and implementation,

including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language provides four primitive types, chosen to cover the core domains of numeric computation, logical control flow, and text manipulation while keeping the type system tractable and the semantics well-defined.

2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 20.5. Float values are used in arithmetic expressions and also arise as the result of any division. Division always produces a Float even when both operands are Integer: for example, `10 / 2` evaluates to `5.0 : Float`. This is an explicit design rule enforced in the type checker.

2.1.3 *Boolean*

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 *String*

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At runtime, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 Void

Although the language exposes exactly four programmer-visible types, the implementation introduces a fifth value, `Void`, within the `LanguageType` enumeration. Its sole purpose is to provide a well-defined return type for functions that contain no `return` statement, allowing the symbol table to record a consistent entry for every function regardless of whether it produces a value. `Void` cannot appear in source code and is never visible to the programmer; it is a bookkeeping device used internally by the type checker.

2.2 Static Typing

The language employs static typing, meaning that type errors are detected at type-checking time rather than at runtime. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
1 x = 5;
2 x = "hello";    (* TypeError: Cannot assign String to variable 'x' of
   type Integer *)
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
INT_LITERAL	0, 10, 42
FLOAT_LITERAL	3.14, 20.5
BOOL_LITERAL	true, false
STRING_LITERAL	"hello"
IDENTIFIER	variable and function names such as x, add
Arithmetic operators (integer)	+, -, *, /
Float arithmetic operators	+, -, *, / .
Comparison operators	==, !=
Assignment operator	=
Keywords	if, else, while, def, return, print
Delimiters	() { } , ;

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol program. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```

1 program  -> statement* EOF
2 statement -> assignment
3           | if_stmt
4           | while_stmt
5           | func_def
6           | print_stmt
7           | return_stmt
8           | expr ';'

```

2.4.2 Arithmetic Expressions

```

1 expr  -> term (( '+' | '-' | '+.' | '-.') term)*
2 term  -> factor (( '*' | '/' | '*.' | '/.') factor)*
3 factor -> '-' factor      (* integer unary negation *)
4       | '-.' factor      (* float unary negation *)
5       | INT_LITERAL
6       | FLOAT_LITERAL
7       | STRING_LITERAL
8       | BOOL_LITERAL
9       | IDENTIFIER
10      | IDENTIFIER '(' args? ')',
11      | '(' expr ')',

```

The operators `+`, `-`, `*`, and `/` operate exclusively on Integer operands. The dot-suffixed operators `+.` , `-.`, `*.`, and `/.` operate exclusively on Float operands. This follows the Caml Light convention of using distinct operator tokens for integer and float arithmetic, eliminating the need for implicit numeric promotion. Unary negation is similarly split: `-x` negates an Integer (desugared to `0 - x`), and `-.x` negates a Float (desugared to `0.0 -. x`).

2.4.3 Boolean Expressions

```

1 bool_expr -> expr '==' expr
2           | expr '!=' expr
3           | BOOL_LITERAL
4           | expr (* fallback: type checker enforces Boolean *)

```

Boolean expressions are only parsed as conditions inside `if` and `while` statements. Comparison results are not storable in variables directly; the grammar does not include `bool_expr` in the `factor` rule. This is consistent with the assignment specification, which describes Boolean expressions primarily as conditions.

If `bool_expr` parses an expression and finds no following `==` or `!=` operator, the expression is returned as-is and the type checker verifies that its inferred type is Boolean.

2.4.4 Statements

```

1 assignment -> IDENTIFIER '=' expr ';'
2 if_stmt    -> 'if' '(' bool_expr ')' block ('else' block)?
3 while_stmt -> 'while' '(' bool_expr ')' block
4 print_stmt -> 'print' '(' expr ')' ';'
5 return_stmt -> 'return' expr ';'
6 block      -> '{' statement* '}'

```

2.4.5 Function Definitions and Calls

```

1 func_def -> 'def' IDENTIFIER '(' params? ')' block
2 params   -> IDENTIFIER (',' IDENTIFIER)*
3 args     -> expr (',' expr)*

```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function’s local environment.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest

precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	<code>==</code> , <code>!=</code>	non-associative (condition use only)
2	<code>+</code> , <code>-</code> , <code>+</code> , <code>.</code> , <code>-</code>	left-associative
3 (highest)	<code>*</code> , <code>/</code> , <code>*</code> , <code>.</code> , <code>/</code>	left-associative

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative <code>TokenType</code>
Literals	42, 3.14, true, "hello"	INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, STRING_LITERAL
Identifiers	x, counter, add	IDENTIFIER
Keywords	if, else, while, def, return, print	keyword-specific token variants
Operators	+, -, *, /, =, ==, !=	PLUS, MINUS, STAR, SLASH, ASSIGN, EQUAL_EQUAL, BANG_EQUAL
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	EOF

Token types (from class `TokenType`) are defined using an Enum with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters, and a special end-of-file (EOF) token. The EOF token is appended after all input is processed,

allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
1 @dataclass(frozen=True)
2 class Token:
3     token_type: TokenType
4     lexeme: str
5     line: int
6     column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an EOF token is added at the end.

Whitespace is skipped, while invalid input results in a `LexerError`.

```
1 def tokenize(self) -> list[Token]:
2     tokens: list[Token] = []
3     while not self._is_at_end():
4         self.start = self.current
5         self.token_column = self.column
6         token = self._scan_token()
7         if token is not None:
8             tokens.append(token)
9     tokens.append(Token(TokenType.EOF, "", self.line, self.column))
10    return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers.

```
1 KEYWORDS: dict[str, TokenType] = {
2     "if": TokenType.IF,
3     "else": TokenType.ELSE,
4     "while": TokenType.WHILE,
5     # ...
6     "true": TokenType.BOOL_LITERAL,
7     "false": TokenType.BOOL_LITERAL,
8 }
```

```
1 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
2     "+": TokenType.PLUS,
3     "-": TokenType.MINUS,
4     "*": TokenType.STAR,
5     "/": TokenType.SLASH,
6     # ...
7 }
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
<code>tokenize()</code>	Iterates through source, scans tokens, appends EOF	ordered token list
<code>_scan_token()</code>	Dispatches recognition based on character and lookahead	next token or skip
<code>_number()</code>	Parses numeric sequences, distinguishes int vs float	numeric literal token
<code>_string()</code>	Parses quoted text, checks termination	string token or error
<code>_identifier()</code>	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
KEYWORDS map	Maps reserved words to token types	token type selection
SINGLE_CHAR_TOKENS map	Maps single-character symbols	token type selection
<code>_advance()</code> , <code>_peek()</code> , <code>_match()</code>	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```

1 def lookup(self, name: str) -> Symbol:
2     symbol = self._symbols.get(name)
3     if symbol is not None:
4         return symbol
5     if self.parent is not None:
6         return self.parent.lookup(name)
7     raise SymbolTableError(f"Undefined symbol: {name}")

```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as Integer, Float, Boolean, String, and Void. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
LanguageType enum	Integer, Float, Boolean, String, Void	Defines the type system
Symbol (base)	Name and declared or inferred type	Base abstraction for entries
VariableSymbol	Variable metadata (including initialization state)	Tracks correct usage before assignment
FunctionSymbol	Parameters and return type	Validates calls and return consistency
lookup(name)	Recursive scope search	Resolves identifiers or reports undefined
SymbolTableError	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```

1 def run_pipeline(source_code: str) -> PipelineResult:
2     tokens = Lexer(source_code).tokenize()
3     tree = Parser(tokens).parse()
4     ast_text = ASTPrinter().print(tree)
5     checked_scope = TypeChecker().check(tree)
6     outputs: list[str] = []
7     Interpreter(output_callback=outputs.append).execute(tree)
8     return PipelineResult(
9         tokens=tokens,
10        ast_text=ast_text,
11        checked_scope=checked_scope,
12        outputs=outputs,
13    )

```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser and the associated abstract syntax tree (AST) model. Starting from lexer-produced tokens, the parser constructs a hierarchical Program AST that represents expressions, statements, and control-flow structure in a form suitable for downstream semantic analysis and execution. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` statements. Equality operators (`==`, `!=`) are parsed in condition contexts (if/while) through the parser’s boolean-expression path.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (line, column). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for
4         diagnostics."""
5     line: int = field(default=0, kw_only=True)
6     column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
1 if tok.token_type == TokenType.STRING_LITERAL:
2     self._advance()
3     return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
1 @dataclass
2 class BinaryOp(ASTNode):
3     """An arithmetic or comparison binary operation.
4
5     op is one of: '+', '-', '*', '/', '==', '!=',
6     Example:  a + b    x == 0
7     """
8     op: str
9     left: ASTNode
10    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Print` and `Return` nodes each store a single expression and represent output and function return operations respectively. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the

AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single Program AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
1 def parse(self) -> Program:
2     """Parse the full token stream and return the root Program node."""
3     line, col = self._peek().line, self._peek().column
4     statements: list[ASTNode] = []
5     while not self._is_at_end():
6         statements.append(self._statement())
7     return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, print, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Statement parsing

The main parsing loop repeatedly processes statements until EOF, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, returns, and prints each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```

1 # assignment: IDENTIFIER "=" expr ";"
2 if (
3     tok.token_type == TokenType.IDENTIFIER
4     and self._peek_next().token_type == TokenType.ASSIGN
5 ):
6     return self._assignment()
7 # fallback: bare expression statement expr ";"
8 node = self._expr()
9 self._expect(TokenType.SEMICOLON, "Expected ';' after expression")

```

4.2.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels: `_expr` handles `+` and `-`, `_term` handles `*` and `/`, and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Boolean conditions for `if` and `while` are parsed by `_bool_expr`, which recognises `==` and `!=` after parsing the left-hand arithmetic expression. As a result, equality parsing is condition-focused in the current grammar implementation rather than a general infix operator available in every expression position. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code.

```

1 def _expr(self) -> ASTNode: # +, -
2     ...
3 def _term(self) -> ASTNode: # *, /
4     ...
5 def _factor(self) -> ASTNode: # unary -, literals, identifiers/calls,
6     grouped expr

```

Figure 4.1 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` handles `*` before `_expr` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

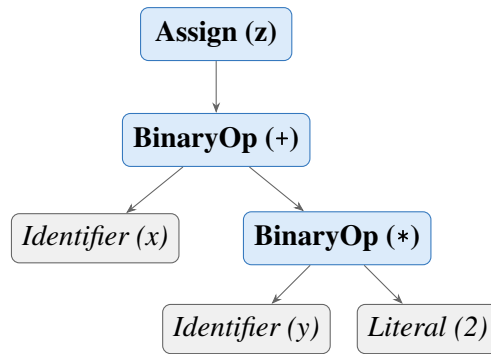


Figure 4.1. AST for $z = x + y * 2$; . Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Boolean conditions for `if` and `while` are parsed through `_bool_expr`, which accepts equality and inequality comparisons but can also return a non-comparison expression node; semantic enforcement that a condition is truly Boolean is deferred to the type checker.

4.2.3 *Helper functions and lookahead*

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token; comparison operators are recognised after parsing the left expression in `_bool_expr`. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.3 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure:

Program and Block include statement counts, BinaryOp explicitly shows left and right subtrees, and FunctionCall prints each argument by index. Literal includes both value and runtime Python type names for clarity.

4.4 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations, while type correctness and semantic constraints are checked later by the type checker. For example, `_bool_expr` may syntactically accept a non-Boolean expression as a condition node, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \text{Identifier} \rightarrow \text{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Type Inference Procedure

The type checker performs type inference by traversing the abstract syntax tree before interpretation. The algorithm is syntax-directed: each AST node is visited, the types of its child nodes are inferred first, and the current node is checked according to the language typing rules.

Algorithm TYPECHECK(*node*, Γ)

Input: *node* = current AST node; Γ = current type environment / symbol table

Output: inferred type of *node*, or `TypeError`

1. **If** *node* is a **Literal**: return the corresponding primitive type: `Integer`, `Float`, `Boolean`, or `String`.
2. **If** *node* is an **Identifier**: look up the identifier in Γ . If it is not found, report an undefined variable error. Otherwise, return its stored type.
3. **If** *node* is a **BinaryOp**:
 - Infer the type of the left operand and the right operand.
 - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Integer`; return `Integer`.
 - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Float`; return `Float`.
 - If the operator is `==` or `!=`: require both operands to be arithmetic; return `Boolean`.
 - Otherwise, report a `TypeError`.
4. **If** *node* is an **Assignment**:
 - Infer the type of the right-hand-side expression.
 - If the variable is not yet in Γ : insert it with the inferred type.
 - Else: verify the new type matches the existing type; if not, report a `TypeError`.
5. **If** *node* is an **If** statement:
 - Infer the condition type; require it to be `Boolean`.
 - Type-check the then-block in a child environment.
 - If an else-block exists, type-check it in a child environment.
6. **If** *node* is a **While** statement:
 - Infer the condition type; require it to be `Boolean`.
 - Type-check the loop body in a child environment.
7. **If** *node* is a **FunctionDef**:
 - Register the function name in Γ .
 - Parameter types are initially unknown.
 - Return type is inferred from return statements in the body.
8. **If** *node* is a **FunctionCall**:

- Infer all argument types.
 - If this is the first valid call: store the argument types as the function’s parameter types.
 - Else: compare argument types with the stored parameter types.
 - Type-check the function body using a local environment.
 - Return the inferred function return type.
9. If *node* is a **Print** statement: infer the expression type; accept any of the four supported types.

5.4 Inference Rules

5.4.1 Literals

Literal nodes have direct types:

$$\begin{aligned}\Gamma \vdash 42 & : \text{Integer} \\ \Gamma \vdash 3.14 & : \text{Float} \\ \Gamma \vdash \text{true} & : \text{Boolean} \\ \Gamma \vdash \text{“hi”} & : \text{String}\end{aligned}$$

5.4.2 No Operator Overloading

The language does not support operator overloading. Following the Caml Light convention, integer and float arithmetic use entirely separate operator tokens, so every operator has a single, fixed type signature:

- $+$, $-$, $*$, $/$ — require two `Integer` operands; result is `Integer`.
- $+. , - . , * . , / .$ — require two `Float` operands; result is `Float`.
- $+$ is not overloaded for string concatenation; applying it to a `String` is a type error.
- $==$ and $!=$ are not overloaded for `Boolean` or `String` equality.
- No implicit numeric promotion: `Integer + Float` is a type error.

Because every operator has a unique, unambiguous type derivable from its token alone, the compiler can always infer expression types without requiring explicit type declarations from the programmer.

5.4.3 Arithmetic Expressions

The integer operators `+`, `-`, `*`, and `/` require both operands to be `Integer` and always return `Integer` (including division, which uses integer floor division). The float operators `+`, `-`, `*`, and `/` require both operands to be `Float` and always return `Float`. Mixing an `Integer` and a `Float` with any operator is a type error.

Table 5.1. Arithmetic operator type rules

Left operand	Operator	Right operand	Result type
Integer	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Integer	Integer
Float	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Float	Float
Integer	any	Float	TypeError
Float	any	Integer	TypeError
Any	any	String or Boolean	TypeError

Unary negation

Unary integer negation (`-x`) is desugared by the parser to `0 - x`. Unary float negation (`- . x`) is desugared to `0 . 0 - . x`. The type checker therefore applies the standard binary rules in both cases.

5.4.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Both operators require two arithmetic (`Integer` or `Float`) operands; this constraint is a deliberate design choice that keeps comparison semantics unambiguous. Comparisons involving `Boolean` or `String` operands are rejected by the type checker. Although `Boolean` and `String` values exist in the language, equality and inequality are deliberately restricted to arithmetic expressions because the project specification defines basic `Boolean` expressions as equality and inequality between two arithmetic expressions.

5.4.5 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the

environment. If the variable already exists, the new value must have the same type.

If $x \notin \Gamma$ and $\Gamma \vdash e : T$, then $\Gamma, x : T \vdash x = e$

If $x : T \in \Gamma$ and $\Gamma \vdash e : T$, then $\Gamma \vdash x = e$

5.4.6 *If-Then-Else*

The condition of an if statement must be Boolean. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B_{\text{then}} \quad \Gamma'' \vdash B_{\text{else}} \implies \Gamma \vdash \text{if}(c) B_{\text{then}} \text{ else } B_{\text{else}}$$

5.4.7 *While-Loop*

The condition of a while loop must also be Boolean. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B \implies \Gamma \vdash \text{while}(c) B$$

5.4.8 *Function Definition and Call*

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the return statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally outside the scope of this implementation; supporting them would require additional fixed-point inference or explicit recursion handling that is orthogonal to the core type-inference algorithm.

5.5 **Type Error Reporting**

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

[line X, col Y] TypeError: message

Examples include assigning a String to an Integer variable, using a non-Boolean condition in an if statement, or supplying the wrong argument type to a function.

5.6 Example Type Inference Traces

Consider the following program fragment:

```
1 x = 10;  
2 y = 20.5;  
3 z = x + y;  
4 print(z);
```

The checker infers:

- x: Integer
- y: Float
- x + y: Float
- z: Float

For a function example:

```
1 def add(a, b) {  
2     return a + b;  
3 }  
4  
5 result = add(2, 3.5);
```

The first function call infers a: Integer and b: Float. The expression a + b is therefore Float, so the function return type is inferred as Float. The variable result is also inferred as Float.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a four-stage pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

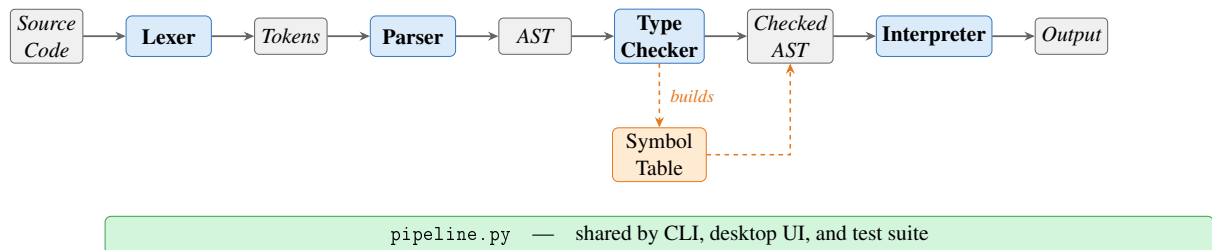


Figure 6.1. Compiler pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All four stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 End-to-end pipeline walkthrough

The following example traces the complete execution of a three-statement program through all four pipeline stages, illustrating the correspondence between source text, tokens, abstract syntax tree, inferred types, and runtime output.

Source program:

```
1 x = 10;  
2 y = 20.5;  
3 print(x + y);
```

Note: in the Caml Light-style design used by this implementation, integer and float arithmetic use separate operators. The equivalent program using the actual language syntax is:

```
1 a = 3;  
2 b = 4;  
3 print(a + b);
```

Stage 1 — Lexer output (token stream):

```
1 IDENTIFIER      'a'      line 1, col 1  
2 ASSIGN          '='      line 1, col 3  
3 INT_LITERAL     '3'      line 1, col 5  
4 SEMICOLON       ';'      line 1, col 6  
5 IDENTIFIER      'b'      line 2, col 1  
6 ASSIGN          '='      line 2, col 3  
7 INT_LITERAL     '4'      line 2, col 5  
8 SEMICOLON       ';'      line 2, col 6  
9 PRINT           'print'   line 3, col 1  
10 LPAREN          '('      line 3, col 6  
11 IDENTIFIER      'a'      line 3, col 7  
12 PLUS           '+'      line 3, col 9  
13 IDENTIFIER      'b'      line 3, col 11  
14 RPAREN          ')'      line 3, col 12  
15 SEMICOLON       ';'      line 3, col 13  
16 EOF
```

Stage 2 — Parser output (abstract syntax tree):

```
1 Program [3 statement(s)]  
2   Assign 'a' =  
3     Literal 3 (int)  
4   Assign 'b' =  
5     Literal 4 (int)  
6   Print  
7     BinaryOp '+'  
8       left:  
9         Identifier 'a'  
10      right:  
11        Identifier 'b'
```

Stage 3 — Type checker output (inferred type environment):

Name	Inferred type
a	Integer
b	Integer
a + b	Integer (Integer + Integer → Integer)

Stage 4 — Interpreter output:

```
7 : Integer
```

The example demonstrates the full data flow: source characters are tokenised by the lexer, structured into an AST by the parser, validated and annotated by the type checker, and finally evaluated by the interpreter to produce typed output.

6.1.2 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, `python3 main.py` runs a script (or built-in sample) and prints the stages in order: tokens, AST text, type table, and execution output. `python3 ui.py` runs the Tkinter workbench: the same pipeline is used internally, with each stage shown in a separate tab so the user can inspect results without altering the underlying flow.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
<code>main.py</code>	Command-line runner for sample code or a script file
<code>ui.py</code>	Desktop UI for editing and running a script
<code>components/tokens.py</code>	Token classes and token categories
<code>components/lexica.py</code>	Lexical analysis
<code>components/ast_nodes.py</code>	AST node definitions
<code>components/parser.py</code>	Recursive-descent parser
<code>components/ast_printer.py</code>	AST renderer for debugging and reporting
<code>components/symbol_table.py</code>	Scoped symbol table and semantic types
<code>components/type_checker.py</code>	Static type checker and inference
<code>components/interpreter.py</code>	Tree-walking interpreter
<code>components/pipeline.py</code>	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
dataclasses	AST and runtime helper structures
tkinter	Desktop graphical interface
unittest	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
1 print(42);           (* 42 : Integer      *)
2 print(3.14);         (* 3.14 : Float      *)
3 print(true);         (* true : Boolean    *)
4 print("plc");        (* "plc" : String   *)
```

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- `if` execution,
- `while` execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. The window is titled *125970-126690-126687-Ass2-PLC* and opens at 1200×760 pixels in a horizontal split layout.

Toolbar. A top toolbar provides three buttons — *Open Script*, *Run*, and *Clear Output* — and a right-aligned status label that reports *Ready*, a loaded filename, or *Run failed*.

Left panel (source editor). A monospaced Text widget with both horizontal and vertical scrollbars allows the user to enter or paste arbitrary source code. The *Open Script* button

populates the editor from a file. The editor is pre-loaded with the built-in sample program on first launch.

Right panel (output tabs). A Notebook widget presents four tabs, each backed by a scrollable read-only Text widget with both horizontal and vertical scrollbars:

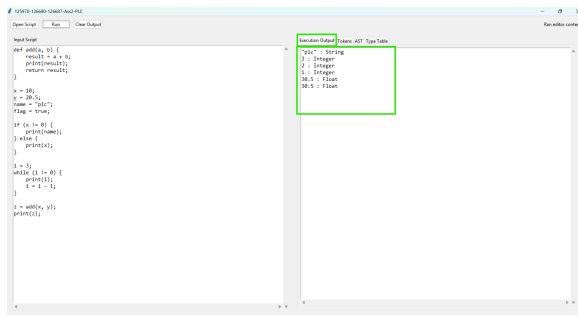
- *Execution Output* — the `print()` values produced by stage 4, formatted as `value : Type`;
- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *AST* — the indented tree text from stage 2;
- *Type Table* — the symbol table from stage 3 showing each name, kind, inferred type, initialisation state, and parameter list.

Error handling. If any pipeline stage raises an exception, the error message is written to the *Execution Output* tab as `ERROR: <message>`. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

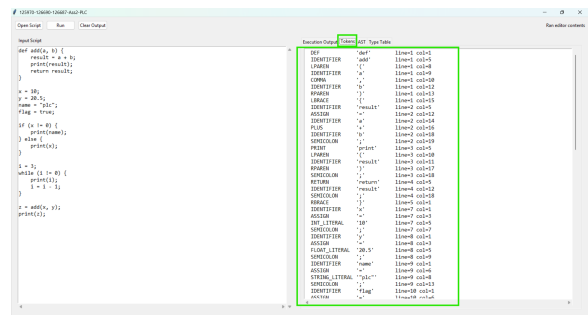
Keyboard shortcuts. F5 and Ctrl+Enter are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

6.6.1 Screenshots

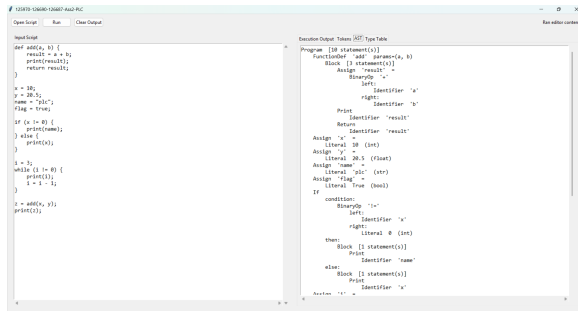
Figures 6.2 and 6.3 show the desktop workbench and CLI output.



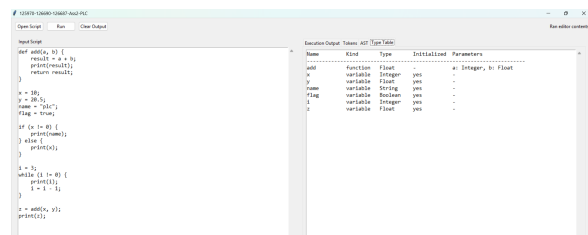
(a) Execution Output tab.



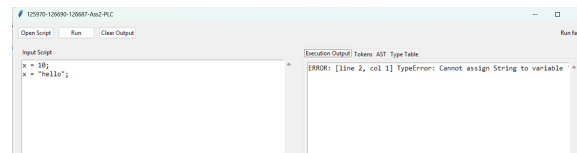
(b) Tokens tab (Stage 1).



(c) AST tab (Stage 2).



(d) Type Table tab (Stage 3).



(e) Inline type-error display.

Figure 6.2. Desktop workbench — output tabs and error display.

6.7 Command-line interface

The CLI runner (main.py) accepts an optional script path; without one it runs the built-in sample and prints four labelled stages to stdout.

```
(.venv) PS C:\Users\Windows\Desktop\125970-126690-126687-Assignment2-PLC> python main.py

=====
STAGE 1 - TOKENS
=====
DEF      'def'      line=2 col=1
IDENTIFIER 'add'     line=2 col=5
LPAREN   '('        line=2 col=8
IDENTIFIER 'a'       line=2 col=9
COMMA    ','        line=2 col=10
IDENTIFIER 'b'       line=2 col=12
RPAREN   ')'        line=2 col=13
LBRACE   '{'        line=2 col=15
IDENTIFIER 'result'  line=3 col=5
ASSIGN   '='        line=3 col=12
IDENTIFIER 'a'       line=3 col=14
PLUS     '+'        line=3 col=16
IDENTIFIER 'b'       line=3 col=18
SEMICOLON ';'        line=3 col=19
PRINT    'print'     line=4 col=5
LPAREN   '('        line=4 col=10
IDENTIFIER 'result'  line=4 col=11
RPAREN   ')'        line=4 col=17
SEMICOLON ';'        line=4 col=18
RETURN   'return'    line=5 col=5
IDENTIFIER 'result'  line=5 col=12
SEMICOLON ';'        line=5 col=18
RBRACE   '}'        line=6 col=1
IDENTIFIER 'x'       line=8 col=1
ASSIGN   '='        line=8 col=3
INT_LITERAL '10'     line=8 col=5
SEMICOLON ';'        line=8 col=7
IDENTIFIER 'y'       line=9 col=1
ASSIGN   '='        line=9 col=3
FLOAT_LITERAL '20.5' line=9 col=5
SEMICOLON ';'        line=9 col=9
IDENTIFIER 'name'    line=10 col=1
ASSIGN   '='        line=10 col=6
STRING_LITERAL 'plc' line=10 col=8
SEMICOLON ';'        line=10 col=13
```

(a) Stage 1: token stream.

```
Assign 'i' =
  BinaryOp '-'
  left:
    Identifier 'i'
  right:
    Literal 1 (int)

Assign 'z' =
  FunctionCall 'add' [2 arg(s)]
  arg[0]:
    Identifier 'x'
  arg[1]:
    Identifier 'y'

Print
  Identifier 'z'

=====
STAGE 3 - TYPE CHECK
=====
Name      Kind      Type      Initialized Parameters
-----
add       function   Float     -           a: Integer, b: Float
x         variable   Integer   yes          -
y         variable   Float     yes          -
name      variable   String    yes          -
flag      variable   Boolean   yes          -
i         variable   Integer   yes          -
z         variable   Float     yes          -

=====
STAGE 4 - EXECUTION
=====
"plc" : String
3 : Integer
2 : Integer
1 : Integer
30.5 : Float
30.5 : Float
(.venv) PS C:\Users\Windows\Desktop\125970-126690-126687-Assignment2-PLC>
```

(b) Stage 3 type table and Stage 4 output.

Figure 6.3. CLI run of python main.py on the built-in sample.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Manual Testing

Prior to the construction of the automated suite, each language feature was exercised manually by running the full pipeline and inspecting stage outputs. Table 7.1 summarises the categories covered.

Table 7.1. Representative manually tested categories

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate mixed numeric expressions	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true`, `false`, and reserved words are emitted with their keyword token types while non-keyword names produce `IDENTIFIER` tokens. For example, the source fragment `x = 10;` produces the token sequence:

```

1 IDENTIFIER  'x'      line=1 col=1
2 ASSIGN      '='      line=1 col=3
3 INT_LITERAL '10'     line=1 col=5
4 SEMICOLON   ';'      line=1 col=7

```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source

position and a message suggesting !=, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 *Parser and AST*

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr/_term/_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5`; produces an `Assign` node while `add(1,2)`; produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello" : String` at runtime (the interpreter re-adds the quotes when formatting string output),
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError`: messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested with integer-only and mixed integer–float programs to verify operator precedence and numeric promotion. The expression `x + y * 2` confirmed that multiplication binds more tightly than addition, and mixed-type operands consistently produced `Float` results in accordance with the promotion rules defined in Chapter 5.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker correctly enforced that both operands of `==` and `!=` must be arithmetic; supplying a `Boolean` or `String` operand produced a source-positioned `TypeCheckError` before any execution.

7.1.5 Assignment Statements

Assignments were tested for both first-use type inference and reassignment consistency. Assigning 5 to a variable and subsequently assigning "hello" to the same variable correctly produced a type error, confirming that the inferred type is fixed after first assignment.

7.1.6 If-Then-Else

Conditional programs were constructed to verify that exactly one branch executes for a given condition and that neither branch is entered when the condition evaluates to the opposite truth value. Non-Boolean conditions were also supplied and confirmed to be rejected by the type checker before execution.

7.1.7 While-Loop

While-loops were tested with an integer countdown from three to zero. The results confirmed that the loop body executes once per iteration and terminates when the condition becomes false, with `print()` producing one output line per pass rather than accumulating output until loop exit.

7.1.8 Functions

Function tests covered value parameter passing, local scope execution, and return value propagation. The type checker correctly inferred parameter types from the first call site and enforced that signature for all subsequent calls to the same function.

7.1.9 *Print*

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `29.5 : Float` and `"plc" : String`.

7.1.10 *Unary Minus*

The unary minus operator was verified for integer literals (`x = -5`), float literals (`y = -3.14`), negated variable references (`y = -x`), and negation inside a larger expression (`x = 3 + -2`). In each case the type checker assigned the correct type and the interpreter produced the expected value.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
1 # Assignment type mismatch
2 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
   Integer
3
4 # Non-Boolean if condition
5 [line 1, col 4] TypeError: If condition must have type Boolean
6
7 # Wrong argument type for function
8 [line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String
```

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.2. Automated test coverage by class

Test class	Stage covered	Tests
LexerTests	Tokenisation (<code>lexica.py</code>)	10
SymbolTableTests	Scoped symbol table (<code>symbol_table.py</code>)	7
ParserTests	Parsing and precedence (<code>parser.py</code>)	5
TypeCheckerTests	Static type inference (<code>type_checker.py</code>)	15
IntegrationTests	Full pipeline execution	17
Total		54

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identifiers, two-character operators) is produced correctly, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 15 tests cover integer and float arithmetic, division producing `Float`, numeric promotion, string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types, both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
python3 -m unittest discover -s tests -v
```

All 54 tests pass.

7.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

CHAPTER 8

CONCLUSION

This project delivered a complete, statically-typed interpreted language supporting Integer, Float, Boolean, and String types, with arithmetic, comparisons, assignments, if-then-else, while-loops, functions with value parameter passing, and a built-in `print()` function.

The system follows a four-stage pipeline: lexer, parser, type checker, and interpreter. Variable and function types are inferred without explicit declarations; programs that violate type rules are rejected before execution. For uncalled functions, the type checker performs a post-pass to infer parameter types from body usage, ensuring type errors are always caught.

The 54 automated tests across five classes — lexical analysis, symbol table, parsing, type checking, and full pipeline — confirm correct behaviour for all language features. Both a command-line runner and a desktop UI are provided.

The implementation demonstrates all core compiler-construction concepts covered in the course — lexical analysis, parsing, static type inference, and tree-walking interpretation — in a complete, tested, and working system. Future work could extend the language with recursive functions, richer data structures, and code generation.

REFERENCES

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
- Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.
- Python Software Foundation. Python documentation: dataclasses — data classes. <https://docs.python.org/3/library/dataclasses.html>, 2026a. Accessed: April 2026.
- Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026b. Accessed: April 2026.
- Python Software Foundation. Python documentation: unittest — unit testing framework. <https://docs.python.org/3/library/unittest.html>, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE

The complete source code is available on GitHub:

<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>

The project can be run from the command line or the desktop UI:

```
1 python3 main.py
2 python3 ui.py
3 python3 -m unittest discover -s tests -v
```

No external Python dependencies are required.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Contributions
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing rules; type checking; assignment, if, while, function, and <code>print()</code> execution; unary minus inference; pipeline integration (<code>pipeline.py</code>); automated test suite (54 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords, operators, identifiers, and literals; symbol table (variable/function/type storage)
Win Htut Naing	st126687	Arithmetic and boolean expressions; assignment, if-then-else, while-loop, function definition/call, and <code>print()</code> parsing; unary minus parsing

A.1 Entry Points

```
1 from __future__ import annotations
2
3 import argparse
4 from pathlib import Path
5
6 from components.pipeline import format_stage_output, run_pipeline
7
8
```

```

9  DEFAULT_SOURCE = """
10 def add(a, b) {
11     result = a + b;
12     print(result);
13     return result;
14 }
15
16 x = 10;
17 y = 20.5;
18 name = "plc";
19 flag = true;
20
21 if (x != 0) {
22     print(name);
23 } else {
24     print(x);
25 }
26
27 i = 3;
28 while (i != 0) {
29     print(i);
30     i = i - 1;
31 }
32
33 z = add(x, y);
34 print(z);
35 """
36
37
38 def run_source(source_code: str) -> None:
39     result = run_pipeline(source_code)
40     print(format_stage_output(result))
41
42
43 def _parse_args() -> argparse.Namespace:
44     parser = argparse.ArgumentParser(description="Run the programming
45         language pipeline.")
46     parser.add_argument("script", nargs="?", help="Optional path to a
47         source file to run")
48     return parser.parse_args()
49
50 def main() -> None:
51     args = _parse_args()
52     if args.script:
53         source_code = Path(args.script).read_text(encoding="utf-8")
54     else:
55         source_code = DEFAULT_SOURCE
56     run_source(source_code)
57
58 if __name__ == "__main__":
59     main()

```

Listing 8.1: main.py — command-line entry point

```
1 from __future__ import annotations
2
3 import tkinter as tk
4 from pathlib import Path
5 from tkinter import filedialog, ttk
6
7 from components.pipeline import format_tokens, run_pipeline
8
9
10 DEFAULT_SOURCE = """def add(a, b) {
11     result = a + b;
12     print(result);
13     return result;
14 }
15
16 x = 10;
17 y = 20.5;
18 name = \"plc\";
19 flag = true;
20
21 if (x != 0) {
22     print(name);
23 } else {
24     print(x);
25 }
26
27 i = 3;
28 while (i != 0) {
29     print(i);
30     i = i - 1;
31 }
32
33 z = add(x, y);
34 print(z);
35 """
36
37
38 class LanguageWorkbench:
39     def __init__(self) -> None:
40         self.root = tk.Tk()
41         self.root.title("125970-126690-126687-Ass2-PLC")
42         self.root.geometry("1200x760")
43         self.current_file: Path | None = None
44
45         self._build_layout()
46         self.source_text.insert("1.0", DEFAULT_SOURCE)
47         self.root.bind("<F5>", lambda _e: self._run_source())
48         self.root.bind("<Control-Return>", lambda _e: self._run_source
49             ())
```

```

50 def _build_layout(self) -> None:
51     toolbar = ttk.Frame(self.root, padding=10)
52     toolbar.pack(fill=tk.X)
53
54     ttk.Button(toolbar, text="Open Script", command=self.
55         _open_script).pack(side=tk.LEFT)
56     ttk.Button(toolbar, text="Run", command=self._run_source).pack(
57         side=tk.LEFT, padx=(8, 0))
58     ttk.Button(toolbar, text="Clear Output", command=self.
59         _clear_output).pack(side=tk.LEFT, padx=(8, 0))
60
61     self.status_var = tk.StringVar(value="Ready")
62     ttk.Label(toolbar, textvariable=self.status_var).pack(side=tk.
63         RIGHT)
64
65     body = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
66     body.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
67
68     left_panel = ttk.Frame(body, padding=8)
69     right_panel = ttk.Frame(body, padding=8)
70     body.add(left_panel, weight=1)
71     body.add(right_panel, weight=1)
72
73     ttk.Label(left_panel, text="Input Script").pack(anchor=tk.W)
74     src_frame = ttk.Frame(left_panel)
75     src_frame.pack(fill=tk.BOTH, expand=True, pady=(6, 0))
76     self.source_text = tk.Text(src_frame, wrap=tk.NONE, font=(
77         "Consolas", 11))
78     src_vsb = ttk.Scrollbar(src_frame, orient=tk.VERTICAL, command=
79         self.source_text.yview)
80     src_hsb = ttk.Scrollbar(src_frame, orient=tk.HORIZONTAL,
81         command=self.source_text.xview)
82     self.source_text.configure(yscrollcommand=src_vsb.set,
83         xscrollcommand=src_hsb.set)
84     src_vsb.pack(side=tk.RIGHT, fill=tk.Y)
85     src_hsb.pack(side=tk.BOTTOM, fill=tk.X)
86     self.source_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
87
88     notebook = ttk.Notebook(right_panel)
89     notebook.pack(fill=tk.BOTH, expand=True)
90
91     self.output_text = self._make_output_tab(notebook, "Execution
92         Output")
93     self.tokens_text = self._make_output_tab(notebook, "Tokens")
94     self.ast_text = self._make_output_tab(notebook, "AST")
95     self.types_text = self._make_output_tab(notebook, "Type Table")
96
97 def _make_output_tab(self, notebook: ttk.Notebook, title: str) ->
98     tk.Text:
99     frame = ttk.Frame(notebook, padding=8)
100     notebook.add(frame, text=title)
101     text = tk.Text(frame, wrap=tk.NONE, font=("Consolas", 11),
102         state=tk.DISABLED)

```

```

92     vsb = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=text.
93         yview)
94     hsb = ttk.Scrollbar(frame, orient=tk.HORIZONTAL, command=text.
95         xview)
96     text.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
97     vsb.pack(side=tk.RIGHT, fill=tk.Y)
98     hsb.pack(side=tk.BOTTOM, fill=tk.X)
99     text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
100     return text
101
102 def _open_script(self) -> None:
103     file_path = filedialog.askopenfilename(
104         title="Open Script File",
105         filetypes=[("Text Files", "*.txt *.pl *.src"), ("All Files"
106             , "*.*)"],
107     )
108     if not file_path:
109         return
110
111     path = Path(file_path)
112     self.current_file = path
113     self.source_text.delete("1.0", tk.END)
114     self.source_text.insert("1.0", path.read_text(encoding="utf-8")
115         )
116     self.status_var.set(f"Loaded {path.name}")
117
118 def _run_source(self) -> None:
119     source = self.source_text.get("1.0", tk.END)
120     try:
121         result = run_pipeline(source)
122     except Exception as error:
123         self._write_text(self.output_text, f"ERROR: {error}")
124         self.status_var.set("Run failed")
125         return
126
127     self._write_text(self.output_text, "\n".join(result.outputs) if
128         result.outputs else "(no output)")
129     self._write_text(self.tokens_text, format_tokens(result.tokens)
130         )
131     self._write_text(self.ast_text, result.ast_text)
132     self._write_text(self.types_text, result.checked_scope.
133         format_table())
134
135     current_label = self.current_file.name if self.current_file is
136         not None else "editor contents"
137     self.status_var.set(f"Ran {current_label}")
138
139 def _clear_output(self) -> None:
140     self._write_text(self.output_text, "")
141     self._write_text(self.tokens_text, "")
142     self._write_text(self.ast_text, "")
143     self._write_text(self.types_text, "")
144     self.status_var.set("Output cleared")

```

```

137
138     @staticmethod
139     def _write_text(widget: tk.Text, content: str) -> None:
140         widget.config(state=tk.NORMAL)
141         widget.delete("1.0", tk.END)
142         widget.insert("1.0", content)
143         widget.config(state=tk.DISABLED)
144
145     def run(self) -> None:
146         self.root.mainloop()
147
148
149 if __name__ == "__main__":
150     LanguageWorkbench().run()

```

Listing 8.2: ui.py — Tkinter desktop workbench

A.2 Core Components

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from enum import Enum, auto
5
6
7 class TokenType(Enum):
8     # Literals
9     INT_LITERAL = auto()
10    FLOAT_LITERAL = auto()
11    BOOL_LITERAL = auto()
12    STRING_LITERAL = auto()
13
14    # Identifier
15    IDENTIFIER = auto()
16
17    # Keywords
18    IF = auto()
19    ELSE = auto()
20    WHILE = auto()
21    DEF = auto()
22    RETURN = auto()
23    PRINT = auto()
24
25    # Operators
26    PLUS = auto()
27    MINUS = auto()
28    STAR = auto()
29    SLASH = auto()
30    PLUS_DOT = auto()    # +. (float addition)
31    MINUS_DOT = auto()   # -. (float subtraction)
32    STAR_DOT = auto()    # *. (float multiplication)
33    SLASH_DOT = auto()   # /. (float division)

```

```

34     EQUAL_EQUAL = auto()
35     BANG_EQUAL = auto()
36     ASSIGN = auto()
37
38     # Delimiters
39     LPAREN = auto()
40     RPAREN = auto()
41     LBRACE = auto()
42     RBRACE = auto()
43     COMMA = auto()
44     SEMICOLON = auto()
45
46     EOF = auto()
47
48
49 @dataclass(frozen=True)
50 class Token:
51     token_type: TokenType
52     lexeme: str
53     line: int
54     column: int

```

Listing 8.3: components/tokens.py — token type enumeration and Token dataclass

```

1 from __future__ import annotations
2
3 from typing import Iterator
4
5 from components.tokens import Token, TokenType
6
7
8 KEYWORDS: dict[str, TokenType] = {
9     "if": TokenType.IF,
10    "else": TokenType.ELSE,
11    "while": TokenType.WHILE,
12    "def": TokenType.DEF,
13    "return": TokenType.RETURN,
14    "print": TokenType.PRINT,
15    "true": TokenType.BOOL_LITERAL,
16    "false": TokenType.BOOL_LITERAL,
17 }
18
19
20 # Arithmetic operators that may have a trailing '.' for the float
variant
21 _ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
22     "+": (TokenType.PLUS, TokenType.PLUS_DOT),
23     "-": (TokenType.MINUS, TokenType.MINUS_DOT),
24     "*": (TokenType.STAR, TokenType.STAR_DOT),
25     "/": (TokenType.SLASH, TokenType.SLASH_DOT),
26 }
27
28 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {

```

```

29     "=": TokenType.ASSIGN,
30     "(": TokenType.LPAREN,
31     ")": TokenType.RPAREN,
32     "{": TokenType.LBRACE,
33     "}": TokenType.RBRACE,
34     ",": TokenType.COMMA,
35     ";": TokenType.SEMICOLON,
36 }
37
38
39 class LexerError(Exception):
40     pass
41
42
43 class Lexer:
44     def __init__(self, source: str) -> None:
45         self.source = source
46         self.start = 0
47         self.current = 0
48         self.line = 1
49         self.column = 1
50         self.token_column = 1
51
52     def tokenize(self) -> list[Token]:
53         tokens: list[Token] = []
54         while not self._is_at_end():
55             self.start = self.current
56             self.token_column = self.column
57             token = self._scan_token()
58             if token is not None:
59                 tokens.append(token)
60         tokens.append(Token(TokenType.EOF, "", self.line, self.column))
61         return tokens
62
63     def _scan_token(self) -> Token | None:
64         char = self._advance()
65
66         # Whitespace handling
67         if char in {" ", "\t", "\r"}:
68             return None
69         if char == "\n":
70             self.line += 1
71             self.column = 1
72             return None
73
74         # Two-char operators
75         if char == "=":
76             if self._match("=="):
77                 return self._make_token(TokenType.EQUAL_EQUAL)
78             return self._make_token(TokenType.ASSIGN)
79         if char == "!":
80             if self._match("!="):
81                 return self._make_token(TokenType.BANG_EQUAL)

```

```

82         raise LexerError(self._error_message("Unexpected '!' . Did
83             you mean '!= '?'"))
84
85     # Arithmetic operators: + - * / (with optional trailing '.'
86     # for float variant)
87     arith = _ARITH_DOT.get(char)
88     if arith is not None:
89         int_tok, float_tok = arith
90         # Unary-minus lookahead: '-' followed by a digit is NOT
91         # '-' even if
92         # next char is '.' -- we still need to check char after '.'
93         # is not digit
94         # (to avoid consuming the '.' of a float literal like 3.14)
95         # Rule: X. is a float operator only when the char after '.'
96         # is NOT a digit.
97         if self._peek() == "." and not self._peek_next().isdigit():
98             self._advance() # consume '.'
99             return self._make_token(float_tok)
100         return self._make_token(int_tok)
101
102     # Single-char operators/delimiters
103     single_char_token = SINGLE_CHAR_TOKENS.get(char)
104     if single_char_token is not None:
105         return self._make_token(single_char_token)
106
107     if char == '"':
108         return self._string()
109
110     if char.isdigit():
111         return self._number()
112
113     if self._is_identifier_start(char):
114         return self._identifier()
115
116     raise LexerError(self._error_message(f"Unexpected character: {
117         char!r}"))
118
119 def _string(self) -> Token:
120     while self._peek() != '"' and not self._is_at_end():
121         if self._peek() == "\n":
122             self.line += 1
123             self.column = 1
124             self._advance()
125
126     if self._is_at_end():
127         raise LexerError(self._error_message("Unterminated string
128             literal"))
129
130     # Closing quote
131     self._advance()
132     return self._make_token(TokenType.STRING_LITERAL)

```

```

127 def _number(self) -> Token:
128     while self._peek().isdigit():
129         self._advance()
130
131     is_float = False
132     if self._peek() == "." and self._peek_next().isdigit():
133         is_float = True
134         self._advance()
135         while self._peek().isdigit():
136             self._advance()
137
138     if is_float:
139         return self._make_token(TokenType.FLOAT_LITERAL)
140     return self._make_token(TokenType.INT_LITERAL)
141
142 def _identifier(self) -> Token:
143     while self._is_identifier_part(self._peek()):
144         self._advance()
145
146     lexeme = self._current_lexeme()
147     token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
148     return self._make_token(token_type)
149
150 def _current_lexeme(self) -> str:
151     return self.source[self.start : self.current]
152
153 def _make_token(self, token_type: TokenType) -> Token:
154     return Token(token_type, self._current_lexeme(), self.line,
155                 self.token_column)
156
157 def _advance(self) -> str:
158     char = self.source[self.current]
159     self.current += 1
160     self.column += 1
161     return char
162
163 def _match(self, expected: str) -> bool:
164     if self._is_at_end():
165         return False
166     if self.source[self.current] != expected:
167         return False
168
169     self.current += 1
170     self.column += 1
171     return True
172
173 def _peek(self) -> str:
174     if self._is_at_end():
175         return "\0"
176     return self.source[self.current]
177
178 def _peek_next(self) -> str:
179     if self.current + 1 >= len(self.source):

```

```

179         return "\0"
180         return self.source[self.current + 1]
181
182     def _is_at_end(self) -> bool:
183         return self.current >= len(self.source)
184
185     @staticmethod
186     def _is_identifier_start(char: str) -> bool:
187         return char.isalpha() or char == "_"
188
189     @staticmethod
190     def _is_identifier_part(char: str) -> bool:
191         return char.isalnum() or char == "_"
192
193     def _error_message(self, message: str) -> str:
194         return f"[line {self.line}, col {self.token_column}] {message}"
195
196
197 def iter_tokens(source: str) -> Iterator[Token]:
198     yield from Lexer(source).tokenize()

```

Listing 8.4: components/lexica.py — lexical analyser

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from typing import Optional
5
6
7 #
8     -----
9
10 # Base
11 #
12     -----
13
14 @dataclass
15 class ASTNode:
16     """Base class for every AST node. Carries source location for
17     diagnostics."""
18     line: int = field(default=0, kw_only=True)
19     column: int = field(default=0, kw_only=True)
20
21 #
22     -----
23
24 # Expressions
25 #
26     -----

```

```

22 @dataclass
23 class Literal(ASTNode):
24     """An integer, float, boolean, or string literal.
25
26     Examples:  42    3.14    true    "hello"
27     """
28     value: int | float | bool | str
29
30
31 @dataclass
32 class Identifier(ASTNode):
33     """A variable or function name used as an expression.
34
35     Example:  x
36     """
37     name: str
38
39
40 @dataclass
41 class BinaryOp(ASTNode):
42     """An arithmetic or comparison binary operation.
43
44     op is one of: '+' '-' '*' '/' '==' '!='
45     Example:  a + b    x == 0
46     """
47     op: str
48     left: ASTNode
49     right: ASTNode
50
51
52 @dataclass
53 class FunctionCall(ASTNode):
54     """A call to a user-defined function.
55
56     Example:  add(1, 2)
57     """
58     name: str
59     args: list[ASTNode] = field(default_factory=list)
60
61
62 #
63 # -----
64 #
65
66 # Statements
67 # -----
68
69
70 @dataclass
71 class Assign(ASTNode):
72     """Variable assignment (first use infers type; later uses must
73     match).

```

```

70     Example:  x = 10;
71     """
72     name: str
73     value: ASTNode
74
75
76 @dataclass
77 class Print(ASTNode):
78     """Built-in print statement.
79
80     Example:  print(x);
81     """
82     expr: ASTNode
83
84
85 @dataclass
86 class Return(ASTNode):
87     """Return statement inside a function body.
88
89     Example:  return result;
90     """
91     expr: ASTNode
92
93
94 @dataclass
95 class Block(ASTNode):
96     """A brace-enclosed sequence of statements.
97
98     Example:  { x = 1; print(x); }
99     """
100     statements: list[ASTNode] = field(default_factory=list)
101
102
103 @dataclass
104 class If(ASTNode):
105     """If / optional else control flow.
106
107     Example:  if (x != 0) { print(x); } else { print(0); }
108     """
109     condition: ASTNode
110     then_block: Block
111     else_block: Optional[Block]
112
113
114 @dataclass
115 class While(ASTNode):
116     """While loop.
117
118     Example:  while (x != 0) { x = x - 1; }
119     """
120     condition: ASTNode
121     body: Block
122

```

```

123
124 @dataclass
125 class FunctionDef(ASTNode):
126     """Function definition.
127
128     Example:  def add(a, b) { return a + b; }
129
130     Note: parameter types are unknown at parse time -- Member 3's type
131           checker
132           will infer them from usage. We store only the parameter names here.
133     """
134     name: str
135     params: list[str]
136     body: Block
137
138 #
139 # -----
140 # Top-level
141 # -----
142
143 @dataclass
144 class Program(ASTNode):
145     """Root node: the entire source file as a list of statements."""
146     statements: list[ASTNode] = field(default_factory=list)

```

Listing 8.5: components/ast_nodes.py — AST node definitions

```

1 from __future__ import annotations
2
3 from typing import Optional
4
5 from components.tokens import Token, TokenType
6 from components.ast_nodes import (
7     ASTNode,
8     Program,
9     Block,
10    Assign,
11    If,
12    While,
13    FunctionDef,
14    FunctionCall,
15    Print,
16    Return,
17    BinaryOp,
18    Literal,
19    Identifier,
20 )
21
22

```

```

23 #
24 # Parser error
25 #
26
27 class ParseError(Exception):
28     pass
29
30
31 #
32 # Parser
33 #
34
35 class Parser:
36     """Recursive-descent parser.
37
38     Usage:
39         tokens = Lexer(source).tokenize()
40         tree   = Parser(tokens).parse()    # returns Program node
41     """
42
43     def __init__(self, tokens: list[Token]) -> None:
44         self._tokens = tokens
45         self._pos = 0
46
47     #
48     # Public entry point
49     #
50
51     def parse(self) -> Program:
52         """Parse the full token stream and return the root Program node
53         ."""
54         line, col = self._peek().line, self._peek().column
55         statements: list[ASTNode] = []
56         while not self._is_at_end():
57             statements.append(self._statement())
58         return Program(statements=statements, line=line, column=col)
59
60     # Statements

```

```

61      #
62
63      def _statement(self) -> ASTNode:
64          """Dispatch to the correct statement parser based on the
65             current token."""
66          tok = self._peek()
67
68          if tok.token_type == TokenType.DEF:
69              return self._func_def()
70
71          if tok.token_type == TokenType.IF:
72              return self._if_stmt()
73
74          if tok.token_type == TokenType.WHILE:
75              return self._while_stmt()
76
77          if tok.token_type == TokenType.RETURN:
78              return self._return_stmt()
79
80          if tok.token_type == TokenType.PRINT:
81              return self._print_stmt()
82
83          # assignment: IDENTIFIER "=" expr ";"
84          if (
85              tok.token_type == TokenType.IDENTIFIER
86              and self._peek_next().token_type == TokenType.ASSIGN
87          ):
88              return self._assignment()
89
90          # fallback: bare expression statement expr ";"
91          node = self._expr()
92          self._expect(TokenType.SEMICOLON, "Expected ';' after
93             expression")
94          return node
95
96      def _assignment(self) -> Assign:
97          """IDENTIFIER '=' expr ';'"""
98          name_tok = self._advance() #
99          IDENTIFIER # '='
100          self._advance() # '='
101          value = self._expr()
102          self._expect(TokenType.SEMICOLON, "Expected ';' after
103             assignment")
104          return Assign(name=name_tok.lexeme, value=value,
105                      line=name_tok.line, column=name_tok.column)
106
107      def _if_stmt(self) -> If:
108          """'if' '(' bool_expr ')' block ( 'else' block )?"""
109          tok = self._advance() # 'if'
110          self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
111          condition = self._bool_expr()

```

```

108     self._expect(TokenType.RPAREN, "Expected ')' after if condition
109         ")
110     then_block = self._block()
111     else_block: Optional[Block] = None
112     if self._check(TokenType.ELSE):
113         self._advance()
114         # 'else
115         else_block = self._block()
116     return If(condition=condition, then_block=then_block,
117         else_block=else_block,
118         line=tok.line, column=tok.column)
119
120 def _while_stmt(self) -> While:
121     """'while' '(' bool_expr ')' block"""
122     tok = self._advance()
123     # '
124     while'
125     self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
126     condition = self._bool_expr()
127     self._expect(TokenType.RPAREN, "Expected ')' after while
128         condition")
129     body = self._block()
130     return While(condition=condition, body=body,
131         line=tok.line, column=tok.column)
132
133 def _func_def(self) -> FunctionDef:
134     """'def' IDENTIFIER '(' params? ')' block"""
135     tok = self._advance()
136     # 'def'
137     name_tok = self._expect(TokenType.IDENTIFIER, "Expected
138         function name after 'def'")
139     self._expect(TokenType.LPAREN, "Expected '(' after function
140         name")
141     params = self._params()
142     self._expect(TokenType.RPAREN, "Expected ')' after parameters")
143     body = self._block()
144     return FunctionDef(name=name_tok.lexeme, params=params, body=
145         body,
146         line=tok.line, column=tok.column)
147
148 def _print_stmt(self) -> Print:
149     """'print' '(' expr ')' ';'"""
150     tok = self._advance()
151     # '
152     print'
153     self._expect(TokenType.LPAREN, "Expected '(' after 'print'")
154     expr = self._expr()
155     self._expect(TokenType.RPAREN, "Expected ')' after print
156         expression")
157     self._expect(TokenType.SEMICOLON, "Expected ';' after print
158         statement")
159     return Print(expr=expr, line=tok.line, column=tok.column)
160
161 def _return_stmt(self) -> Return:
162     """'return' expr ';'"""
163     tok = self._advance()
164     # '

```

```

        return'
150     expr = self._expr()
151     self._expect(TokenType.SEMICOLON, "Expected ';' after return
        value")
152     return Return(expr=expr, line=tok.line, column=tok.column)
153
154 def _block(self) -> Block:
155     """{' statement* '}'"""
156     tok = self._expect(TokenType.LBRACE, "Expected '{'")
157     statements: list[ASTNode] = []
158     while not self._check(TokenType.RBRACE) and not self._is_at_end
        ():
159         statements.append(self._statement())
160     self._expect(TokenType.RBRACE, "Expected '}' to close block")
161     return Block(statements=statements, line=tok.line, column=tok.
        column)
162
163 #
164 # Parameters (function definition)
165 #
166
167 def _params(self) -> list[str]:
168     """IDENTIFIER (',' IDENTIFIER)* -- returns list of param names.
        """
169     params: list[str] = []
170     if self._check(TokenType.IDENTIFIER):
171         params.append(self._advance().lexeme)
172         while self._check(TokenType.COMMA):
173             self._advance() # ','
174             tok = self._expect(TokenType.IDENTIFIER, "Expected
                parameter name after ','")
175             params.append(tok.lexeme)
176     return params
177
178 #
179 # Boolean expressions (lowest precedence, used in if/while
        conditions)
180 #
181
182 def _bool_expr(self) -> ASTNode:
183     """expr ('==' | '!=') expr | BOOL_LITERAL"""
184     # bare boolean literal: true / false
185     if self._check(TokenType.BOOL_LITERAL):
186         tok = self._advance()
187         value = tok.lexeme == "true"

```

```

188         return Literal(value=value, line=tok.line, column=tok.
189                           column)
190
191     left = self._expr()
192
193     if self._check(TokenType.EQUAL_EQUAL):
194         op_tok = self._advance()
195         right = self._expr()
196         return BinaryOp(op="==", left=left, right=right,
197                         line=op_tok.line, column=op_tok.column)
198
199     if self._check(TokenType.BANG_EQUAL):
200         op_tok = self._advance()
201         right = self._expr()
202         return BinaryOp(op="!=", left=left, right=right,
203                         line=op_tok.line, column=op_tok.column)
204
205     # no comparison operator found -- return the expr as-is
206     # (the type checker will validate it's Boolean)
207     return left
208
209     # -----
210
211     # Arithmetic expressions (standard precedence)
212     # -----
213
214
215     def _expr(self) -> ASTNode:
216         """term (('+' | '-' | '+.' | '-.') term)*"""
217         node = self._term()
218         while self._check(TokenType.PLUS) or self._check(TokenType.
219             MINUS) \
220             or self._check(TokenType.PLUS_DOT) or self._check(
221                 TokenType.MINUS_DOT):
222             op_tok = self._advance()
223             right = self._term()
224             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
225                             line=op_tok.line, column=op_tok.column)
226         return node
227
228     def _term(self) -> ASTNode:
229         """factor (('*' | '/' | '*.' | '/.') factor)*"""
230         node = self._factor()
231         while self._check(TokenType.STAR) or self._check(TokenType.
232             SLASH) \
233             or self._check(TokenType.STAR_DOT) or self._check(
234                 TokenType.SLASH_DOT):
235             op_tok = self._advance()
236             right = self._factor()
237             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
238                             line=op_tok.line, column=op_tok.column)

```

```

232     return node
233
234 def _factor(self) -> ASTNode:
235     """
236     '-' factor (unary negation, desugared to 0 - factor)
237     | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
238     | IDENTIFIER ( '(' args? ')' )?
239     | '(' expr ')'
240     """
241     tok = self._peek()
242
243     # Unary minus '-' factor (integer negation, desugared to 0 -
244     # factor)
245     if tok.token_type == TokenType.MINUS:
246         op_tok = self._advance()
247         operand = self._factor()
248         zero = Literal(value=0, line=op_tok.line, column=op_tok.
249             column)
250         return BinaryOp(op="-", left=zero, right=operand,
251             line=op_tok.line, column=op_tok.column)
252
253     # Unary minus-dot '-.' factor (float negation, desugared to
254     # 0.0 -. factor)
255     if tok.token_type == TokenType.MINUS_DOT:
256         op_tok = self._advance()
257         operand = self._factor()
258         zero_f = Literal(value=0.0, line=op_tok.line, column=op_tok
259             .column)
260         return BinaryOp(op="-.", left=zero_f, right=operand,
261             line=op_tok.line, column=op_tok.column)
262
263     # Integer literal
264     if tok.token_type == TokenType.INT_LITERAL:
265         self._advance()
266         return Literal(value=int(tok.lexeme), line=tok.line, column
267             =tok.column)
268
269     # Float literal
270     if tok.token_type == TokenType.FLOAT_LITERAL:
271         self._advance()
272         return Literal(value=float(tok.lexeme), line=tok.line,
273             column=tok.column)
274
275     # Boolean literal
276     if tok.token_type == TokenType.BOOL_LITERAL:
277         self._advance()
278         return Literal(value=(tok.lexeme == "true"), line=tok.line,
279             column=tok.column)
280
281     # String literal (keep the surrounding quotes stripped)
282     if tok.token_type == TokenType.STRING_LITERAL:
283         self._advance()
284         return Literal(value=tok.lexeme[1:-1], line=tok.line,

```

```

        column=tok.column)

278
279 # Identifier -- could be a plain variable or a function call
280 if tok.token_type == TokenType.IDENTIFIER:
281     self._advance()
282     if self._check(TokenType.LPAREN):
283         # function call
284         self._advance() # '('
285         args = self._args()
286         self._expect(TokenType.RPAREN, "Expected ')' after
            function arguments")
287         return FunctionCall(name=tok.lexeme, args=args,
288                             line=tok.line, column=tok.column)
289     return Identifier(name=tok.lexeme, line=tok.line, column=
        tok.column)

290
291 # Grouped expression '(' expr ')'
292 if tok.token_type == TokenType.LPAREN:
293     self._advance() # '('
294     node = self._expr()
295     self._expect(TokenType.RPAREN, "Expected ')' to close
        grouped expression")
296     return node
297
298     raise ParseError(self._error_at(tok, f"Unexpected token: {tok.
        lexeme!r}"))
299
300 #
301 -----
302
303 # Arguments (function call)
304 #
305 -----
306
307 def _args(self) -> list[ASTNode]:
308     """expr (',' expr)*"""
309     args: list[ASTNode] = []
310     if not self._check(TokenType.RPAREN):
311         args.append(self._expr())
312         while self._check(TokenType.COMMA):
313             self._advance() # ','
314             args.append(self._expr())
315         return args
316
317 #
318 -----
319
320 # Token navigation helpers
321 #
322 -----

```

```

318     def _peek(self) -> Token:
319         return self._tokens[self._pos]
320
321     def _peek_next(self) -> Token:
322         if self._pos + 1 < len(self._tokens):
323             return self._tokens[self._pos + 1]
324         return self._tokens[-1] # EOF
325
326     def _advance(self) -> Token:
327         tok = self._tokens[self._pos]
328         if not self._is_at_end():
329             self._pos += 1
330         return tok
331
332     def _check(self, token_type: TokenType) -> bool:
333         return self._peek().token_type == token_type
334
335     def _is_at_end(self) -> bool:
336         return self._peek().token_type == TokenType.EOF
337
338     def _expect(self, token_type: TokenType, message: str) -> Token:
339         if self._check(token_type):
340             return self._advance()
341         raise ParseError(self._error_at(self._peek(), message))
342
343     def _error_at(self, tok: Token, message: str) -> str:
344         return f"[line {tok.line}, col {tok.column}] ParseError: {
            message}"

```

Listing 8.6: components/parser.py — recursive-descent parser

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass, field
4  from enum import Enum
5
6
7  class LanguageType(Enum):
8      INTEGER = "Integer"
9      FLOAT = "Float"
10     BOOLEAN = "Boolean"
11     STRING = "String"
12     VOID = "Void"
13
14
15     @dataclass
16     class Symbol:
17         name: str
18         symbol_type: LanguageType
19
20
21     @dataclass
22     class VariableSymbol(Symbol):

```

```

23     initialized: bool = False
24
25
26 @dataclass
27 class FunctionSymbol(Symbol):
28     parameters: list[tuple[str, LanguageType]] = field(default_factory=
        list)
29
30
31 class SymbolTableError(Exception):
32     pass
33
34
35 class SymbolTable:
36     def __init__(self, parent: SymbolTable | None = None) -> None:
37         self.parent = parent
38         self._symbols: dict[str, Symbol] = {}
39
40     def define_variable(self, name: str, symbol_type: LanguageType,
        initialized: bool = False) -> VariableSymbol:
41         if name in self._symbols:
42             raise SymbolTableError(f"Symbol already defined in this
                scope: {name}")
43         symbol = VariableSymbol(name=name, symbol_type=symbol_type,
            initialized=initialized)
44         self._symbols[name] = symbol
45         return symbol
46
47     def define_function(
48         self,
49         name: str,
50         return_type: LanguageType,
51         parameters: list[tuple[str, LanguageType]],
52     ) -> FunctionSymbol:
53         if name in self._symbols:
54             raise SymbolTableError(f"Symbol already defined in this
                scope: {name}")
55         symbol = FunctionSymbol(name=name, symbol_type=return_type,
            parameters=parameters)
56         self._symbols[name] = symbol
57         return symbol
58
59     def lookup(self, name: str) -> Symbol:
60         symbol = self._symbols.get(name)
61         if symbol is not None:
62             return symbol
63         if self.parent is not None:
64             return self.parent.lookup(name)
65         raise SymbolTableError(f"Undefined symbol: {name}")
66
67     def exists_in_current_scope(self, name: str) -> bool:
68         return name in self._symbols
69

```

```

70 def child_scope(self) -> SymbolTable:
71     return SymbolTable(parent=self)
72
73 def set_initialized(self, name: str) -> None:
74     symbol = self.lookup(name)
75     if not isinstance(symbol, VariableSymbol):
76         raise SymbolTableError(f"Cannot initialize non-variable
77                                 symbol: {name}")
78     symbol.initialized = True
79
80 def snapshot(self) -> dict[str, Symbol]:
81     return dict(self._symbols)
82
83 def format_table(self) -> str:
84     lines = [
85         f"{'Name':<12} {'Kind':<10} {'Type':<10} {'Initialized'
86         ':<12} Parameters",
87         "-" * 72,
88     ]
89
90     for name, symbol in self._symbols.items():
91         if isinstance(symbol, FunctionSymbol):
92             params = ", ".join(f"{param_name}: {param_type.value}"
93                                 for param_name, param_type in symbol.parameters)
94             lines.append(
95                 f"{name:<12} {'function':<10} {symbol.symbol_type.
96                 value:<10} {'-':<12} {params or '-'}"
97             )
98         elif isinstance(symbol, VariableSymbol):
99             initialized = "yes" if symbol.initialized else "no"
100             lines.append(
101                 f"{name:<12} {'variable':<10} {symbol.symbol_type.
102                 value:<10} {initialized:<12} -"
103             )
104         else:
105             lines.append(f"{name:<12} {'symbol':<10} {symbol.
106                             symbol_type.value:<10} {'-':<12} -")
107
108     return "\n".join(lines)

```

Listing 8.7: components/symbol_table.py — scoped symbol table

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from components.ast_nodes import (
6     ASTNode,
7     Assign,
8     BinaryOp,
9     Block,
10    FunctionCall,
11    FunctionDef,

```

```

12     Identifier,
13     If,
14     Literal,
15     Print,
16     Program,
17     Return,
18     While,
19 )
20 from components.symbol_table import FunctionSymbol, LanguageType,
    SymbolTable, SymbolTableError, VariableSymbol
21
22
23 class TypeCheckError(Exception):
24     pass
25
26
27 @dataclass
28 class _FunctionState:
29     node: FunctionDef
30     body_checked: bool = False
31
32
33 @dataclass
34 class _FunctionContext:
35     name: str
36     return_type: LanguageType | None = None
37
38
39 class TypeChecker:
40     def __init__(self) -> None:
41         self.global_scope = SymbolTable()
42         self._functions: dict[str, _FunctionState] = {}
43         self._active_calls: list[str] = []
44
45     def check(self, program: Program) -> SymbolTable:
46         for statement in program.statements:
47             if isinstance(statement, FunctionDef):
48                 self._register_function(statement)
49
50         for statement in program.statements:
51             if isinstance(statement, FunctionDef):
52                 continue
53             self._check_statement(statement, self.global_scope, None)
54
55         self._check_uncalled_functions()
56         return self.global_scope
57
58     def _register_function(self, node: FunctionDef) -> None:
59         if node.name in self._functions:
60             raise TypeCheckError(self._error_at(node, f"Function '{node
                .name}' is already defined"))
61
62         placeholder_parameters = [(param_name, LanguageType.VOID) for

```

```

        param_name in node.params]
63     try:
64         self.global_scope.define_function(node.name, LanguageType.
            VOID, placeholder_parameters)
65     except SymbolTableError as error:
66         raise TypeCheckError(self._error_at(node, str(error))) from
            error
67     self._functions[node.name] = _FunctionState(node=node)
68
69     def _check_statement(
70         self,
71         node: ASTNode,
72         scope: SymbolTable,
73         function_context: _FunctionContext | None,
74     ) -> None:
75         if isinstance(node, Assign):
76             value_type = self._infer_expr_type(node.value, scope)
77             self._assign_variable_type(scope, node, value_type)
78             return
79
80         if isinstance(node, Print):
81             self._infer_expr_type(node.expr, scope)
82             return
83
84         if isinstance(node, Return):
85             if function_context is None:
86                 raise TypeCheckError(self._error_at(node, "'return' is
                    only valid inside a function"))
87             expr_type = self._infer_expr_type(node.expr, scope)
88             if function_context.return_type is None:
89                 function_context.return_type = expr_type
90             elif function_context.return_type != expr_type:
91                 raise TypeCheckError(
92                     self._error_at(
93                         node,
94                         f"Function '{function_context.name}' returns
                            both {function_context.return_type.value}
                            and {expr_type.value}",
95                     )
96                 )
97             return
98
99         if isinstance(node, If):
100             condition_type = self._infer_expr_type(node.condition,
                scope)
101             if condition_type != LanguageType.BOOLEAN:
102                 raise TypeCheckError(self._error_at(node.condition, "If
                    condition must have type Boolean"))
103             self._check_block(node.then_block, scope.child_scope(),
                function_context)
104             if node.else_block is not None:
105                 self._check_block(node.else_block, scope.child_scope(),
                    function_context)

```

```

106         return
107
108     if isinstance(node, While):
109         condition_type = self._infer_expr_type(node.condition,
110                                                 scope)
111         if condition_type != LanguageType.BOOLEAN:
112             raise TypeCheckError(self._error_at(node.condition, "
113                 While condition must have type Boolean"))
114         self._check_block(node.body, scope.child_scope(),
115                           function_context)
116         return
117
118     if isinstance(node, Block):
119         self._check_block(node, scope.child_scope(),
120                           function_context)
121         return
122
123     if isinstance(node, FunctionCall):
124         self._infer_function_call_type(node, scope)
125         return
126
127     raise TypeCheckError(self._error_at(node, f"Unsupported
128         statement node: {type(node).__name__}"))
129
130 def _check_block(
131     self,
132     block: Block,
133     scope: SymbolTable,
134     function_context: _FunctionContext | None,
135 ) -> None:
136     for statement in block.statements:
137         self._check_statement(statement, scope, function_context)
138
139 def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) ->
140 LanguageType:
141     if isinstance(node, Literal):
142         return self._literal_type(node)
143
144     if isinstance(node, Identifier):
145         try:
146             symbol = scope.lookup(node.name)
147         except SymbolTableError as error:
148             raise TypeCheckError(self._error_at(node, str(error)))
149             from error
150         if not isinstance(symbol, VariableSymbol):
151             raise TypeCheckError(self._error_at(node, f"'{node.name
152                 }' is not a variable"))
153         return symbol.symbol_type
154
155     if isinstance(node, FunctionCall):
156         return self._infer_function_call_type(node, scope)
157
158     if isinstance(node, BinaryOp):

```

```

151         left_type = self._infer_expr_type(node.left, scope)
152         right_type = self._infer_expr_type(node.right, scope)
153         return self._infer_binary_type(node, left_type, right_type)
154
155         raise TypeCheckError(self._error_at(node, f"Unsupported
            expression node: {type(node).__name__}"))
156
157     def _infer_function_call_type(self, node: FunctionCall, scope:
SymbolTable) -> LanguageType:
158         function_state = self._functions.get(node.name)
159         if function_state is None:
160             raise TypeCheckError(self._error_at(node, f"Undefined
                function: {node.name}"))
161         if node.name in self._active_calls:
162             raise TypeCheckError(self._error_at(node, f"Recursive
                function calls are not supported: {node.name}"))
163
164         argument_types = [self._infer_expr_type(argument, scope) for
            argument in node.args]
165         function_symbol = self._lookup_function_symbol(node)
166
167         if len(argument_types) != len(function_symbol.parameters):
168             raise TypeCheckError(
169                 self._error_at(
170                     node,
171                     f"Function '{node.name}' expects {len(
                        function_symbol.parameters)} argument(s), got {
                            len(argument_types)}",
172                 )
173             )
174
175         declared_parameter_types = [param_type for _, param_type in
            function_symbol.parameters]
176         if all(param_type == LanguageType.VOID for param_type in
            declared_parameter_types):
177             function_symbol.parameters = list(zip(function_state.node.
                params, argument_types, strict=False))
178         else:
179             for index, ((_, expected_type), actual_type) in enumerate(
                zip(function_symbol.parameters, argument_types, strict=
                    False)):
180                 if expected_type != actual_type:
181                     raise TypeCheckError(
182                         self._error_at(
183                             node.args[index],
184                             f"Argument {index + 1} of '{node.name}'
                                must be {expected_type.value}, got {
                                    actual_type.value}",
185                         )
186                     )
187
188         if not function_state.body_checked:
189             self._check_function_body(function_state, function_symbol)

```

```

190         return function_symbol.symbol_type
191
192     def _check_function_body(self, function_state: _FunctionState,
193                             function_symbol: FunctionSymbol) -> None:
194         function_context = _FunctionContext(name=function_state.node.
195                                             name)
196         function_scope = self.global_scope.child_scope()
197         for parameter_name, parameter_type in function_symbol.
198             parameters:
199             function_scope.define_variable(parameter_name,
200                                             parameter_type, initialized=True)
201
202         self._active_calls.append(function_state.node.name)
203         try:
204             self._check_block(function_state.node.body, function_scope,
205                             function_context)
206         finally:
207             self._active_calls.pop()
208
209         function_symbol.symbol_type = function_context.return_type or
210             LanguageType.VOID
211         function_state.body_checked = True
212
213     def _check_uncalled_functions(self) -> None:
214         """After all call-site checks, type-check any function whose
215            body was never visited."""
216         for name, state in self._functions.items():
217             if not state.body_checked:
218                 try:
219                     function_symbol = self.global_scope.lookup(name)
220                 except SymbolTableError:
221                     continue
222                 if not isinstance(function_symbol, FunctionSymbol):
223                     continue
224                 inferred_params = self._infer_param_types_from_body(
225                     state.node)
226                 function_symbol.parameters = inferred_params
227                 self._check_function_body(state, function_symbol)
228
229     def _infer_param_types_from_body(self, node: FunctionDef) -> list[
230         tuple[str, LanguageType]]:
231         """Infer parameter types by scanning the body for expression
232            contexts.
233
234            Any parameter used as an operand of an arithmetic or comparison
235            operator
236            is inferred as numeric (Float if the sibling operand is a float
237            literal,
238            Integer otherwise). Parameters whose usage gives no type hint
239            default to
240            Integer.
241            """

```

```

230 param_names = set(node.params)
231 inferred: dict[str, LanguageType] = {}
232
233 def scan_expr(n: ASTNode) -> None:
234     if isinstance(n, BinaryOp):
235         if n.op in {"+", "-", "*", "/", "=", "!="}:
236             for operand in (n.left, n.right):
237                 if isinstance(operand, Identifier) and operand.
238                     name in param_names:
239                         inferred.setdefault(operand.name,
240                             LanguageType.INTEGER)
241             elif n.op in {"+.", "-.", "*.", "/."}:
242                 for operand in (n.left, n.right):
243                     if isinstance(operand, Identifier) and operand.
244                         name in param_names:
245                             inferred.setdefault(operand.name,
246                                 LanguageType.FLOAT)
247             scan_expr(n.left)
248             scan_expr(n.right)
249         elif isinstance(n, FunctionCall):
250             for arg in n.args:
251                 scan_expr(arg)
252
253 def scan_stmt(n: ASTNode) -> None:
254     if isinstance(n, Assign):
255         scan_expr(n.value)
256     elif isinstance(n, Print):
257         scan_expr(n.expr)
258     elif isinstance(n, Return):
259         scan_expr(n.expr)
260     elif isinstance(n, If):
261         scan_expr(n.condition)
262         scan_block(n.then_block)
263         if n.else_block:
264             scan_block(n.else_block)
265     elif isinstance(n, While):
266         scan_expr(n.condition)
267         scan_block(n.body)
268     elif isinstance(n, Block):
269         scan_block(n)
270     elif isinstance(n, FunctionCall):
271         for arg in n.args:
272             scan_expr(arg)
273
274 def scan_block(block: Block) -> None:
275     for stmt in block.statements:
276         scan_stmt(stmt)
277
278 scan_block(node.body)
279 return [(param, inferred.get(param, LanguageType.INTEGER)) for
280         param in node.params]
281
282 def _assign_variable_type(self, scope: SymbolTable, node: Assign,

```

```

value_type: LanguageType) -> None:
278     try:
279         symbol = scope.lookup(node.name)
280     except SymbolTableError:
281         scope.define_variable(node.name, value_type, initialized=
            True)
282         return
283
284     if not isinstance(symbol, VariableSymbol):
285         raise TypeCheckError(self._error_at(node, f"'{node.name}',
            is not a variable"))
286
287     if symbol.symbol_type != value_type:
288         raise TypeCheckError(
289             self._error_at(
290                 node,
291                 f"Cannot assign {value_type.value} to variable '{
                    node.name}' of type {symbol.symbol_type.value}",
292             )
293         )
294     symbol.initialized = True
295
296 def _infer_binary_type(
297     self,
298     node: BinaryOp,
299     left_type: LanguageType,
300     right_type: LanguageType,
301 ) -> LanguageType:
302     # Integer operators: both operands must be Integer
303     if node.op in {"+", "-", "*"}:
304         if left_type != LanguageType.INTEGER or right_type !=
            LanguageType.INTEGER:
305             raise TypeCheckError(
306                 self._error_at(node, f"Operator '{node.op}'
                    requires two Integer operands")
307             )
308         return LanguageType.INTEGER
309
310     # Integer division: both operands must be Integer, result is
        Integer
311     if node.op == "/":
312         if left_type != LanguageType.INTEGER or right_type !=
            LanguageType.INTEGER:
313             raise TypeCheckError(
314                 self._error_at(node, f"Operator '{node.op}'
                    requires two Integer operands")
315             )
316         return LanguageType.INTEGER
317
318     # Float operators: both operands must be Float
319     if node.op in {"+.", "-.", "*.", "/."}:
320         if left_type != LanguageType.FLOAT or right_type !=
            LanguageType.FLOAT:
            raise TypeCheckError(

```

```

321         self._error_at(node, f"Operator '{node.op}'
322                             requires two Float operands")
323     )
324     return LanguageType.FLOAT
325
326     if node.op in {"==", "!="}:
327         numeric_comparison = self._is_numeric(left_type) and self.
328             _is_numeric(right_type)
329         if not numeric_comparison:
330             raise TypeCheckError(
331                 self._error_at(
332                     node,
333                     f"Operator '{node.op}' requires two arithmetic
334                         operands",
335                 )
336             )
337         return LanguageType.BOOLEAN
338
339     raise TypeCheckError(self._error_at(node, f"Unsupported
340         operator: {node.op}"))
341
342 def _lookup_function_symbol(self, node: FunctionCall) ->
343     FunctionSymbol:
344     try:
345         symbol = self.global_scope.lookup(node.name)
346     except SymbolTableError as error:
347         raise TypeCheckError(self._error_at(node, str(error))) from
348             error
349     if not isinstance(symbol, FunctionSymbol):
350         raise TypeCheckError(self._error_at(node, f"'{node.name}'
351             is not a function"))
352     return symbol
353
354 @staticmethod
355 def _is_numeric(language_type: LanguageType) -> bool:
356     return language_type in {LanguageType.INTEGER, LanguageType.
357         FLOAT}
358
359 @staticmethod
360 def _literal_type(node: Literal) -> LanguageType:
361     if isinstance(node.value, bool):
362         return LanguageType.BOOLEAN
363     if isinstance(node.value, int):
364         return LanguageType.INTEGER
365     if isinstance(node.value, float):
366         return LanguageType.FLOAT
367     if isinstance(node.value, str):
368         return LanguageType.STRING
369     raise TypeCheckError(f"Unsupported literal value: {node.value!r}
370         ")
371
372 @staticmethod
373 def _error_at(node: ASTNode, message: str) -> str:

```

```
365         return f"[line {node.line}, col {node.column}] TypeError: {  
            message}"
```

Listing 8.8: components/type_checker.py — static type checker with inference

```
1 from __future__ import annotations  
2  
3 from dataclasses import dataclass  
4 from typing import Callable  
5  
6 from components.ast_nodes import (  
7     ASTNode,  
8     Assign,  
9     BinaryOp,  
10    Block,  
11    FunctionCall,  
12    FunctionDef,  
13    Identifier,  
14    If,  
15    Literal,  
16    Print,  
17    Program,  
18    Return,  
19    While,  
20 )  
21 from components.symbol_table import LanguageType  
22  
23  
24 class InterpreterError(Exception):  
25     pass  
26  
27  
28 class _ReturnSignal(Exception):  
29     def __init__(self, value: object) -> None:  
30         self.value = value  
31  
32  
33 @dataclass  
34 class _FunctionRuntime:  
35     node: FunctionDef  
36  
37  
38 class _Environment:  
39     def __init__(self, parent: _Environment | None = None) -> None:  
40         self.parent = parent  
41         self.values: dict[str, object] = {}  
42  
43     def define(self, name: str, value: object) -> None:  
44         self.values[name] = value  
45  
46     def get(self, name: str) -> object:  
47         if name in self.values:  
48             return self.values[name]
```

```

49     if self.parent is not None:
50         return self.parent.get(name)
51     raise InterpreterError(f"Undefined variable: {name}")
52
53 def assign(self, name: str, value: object) -> None:
54     if name in self.values:
55         self.values[name] = value
56         return
57     if self.parent is not None and self.parent.contains(name):
58         self.parent.assign(name, value)
59         return
60     self.values[name] = value
61
62 def contains(self, name: str) -> bool:
63     if name in self.values:
64         return True
65     if self.parent is not None:
66         return self.parent.contains(name)
67     return False
68
69
70 class Interpreter:
71     def __init__(self, output_callback: Callable[[str], None] | None =
None) -> None:
72         self._functions: dict[str, _FunctionRuntime] = {}
73         self._output_callback = output_callback or print
74
75     def execute(self, program: Program) -> None:
76         environment = _Environment()
77         for statement in program.statements:
78             if isinstance(statement, FunctionDef):
79                 self._functions[statement.name] = _FunctionRuntime(node
=statement)
80
81         for statement in program.statements:
82             if isinstance(statement, FunctionDef):
83                 continue
84             self._execute_statement(statement, environment)
85
86     def _execute_statement(self, node: ASTNode, environment:
_Environment) -> None:
87         if isinstance(node, Assign):
88             value = self._evaluate(node.value, environment)
89             environment.assign(node.name, value)
90             return
91
92         if isinstance(node, Print):
93             value = self._evaluate(node.expr, environment)
94             self._output_callback(f"{self._format_value(value)} : {self
._runtime_type(value).value}")
95             return
96
97         if isinstance(node, Return):

```

```

98         raise _ReturnSignal(self._evaluate(node.expr, environment))
99
100     if isinstance(node, If):
101         condition = self._evaluate(node.condition, environment)
102         if not isinstance(condition, bool):
103             raise InterpreterError(self._error_at(node.condition, "
                If condition must evaluate to Boolean"))
104         branch = node.then_block if condition else node.else_block
105         if branch is not None:
106             self._execute_block(branch, _Environment(parent=
                environment))
107         return
108
109     if isinstance(node, While):
110         while True:
111             condition = self._evaluate(node.condition, environment)
112             if not isinstance(condition, bool):
113                 raise InterpreterError(self._error_at(node.
                    condition, "While condition must evaluate to
                    Boolean"))
114             if not condition:
115                 break
116             self._execute_block(node.body, _Environment(parent=
                environment))
117         return
118
119     if isinstance(node, Block):
120         self._execute_block(node, _Environment(parent=environment))
121         return
122
123     if isinstance(node, FunctionCall):
124         self._evaluate(node, environment)
125         return
126
127     raise InterpreterError(self._error_at(node, f"Unsupported
        statement node: {type(node).__name__}"))
128
129     def _execute_block(self, block: Block, environment: _Environment)
130     -> None:
131         for statement in block.statements:
132             self._execute_statement(statement, environment)
133
134     def _evaluate(self, node: ASTNode, environment: _Environment) ->
135     object:
136         if isinstance(node, Literal):
137             return node.value
138
139         if isinstance(node, Identifier):
140             return environment.get(node.name)
141
142         if isinstance(node, BinaryOp):
143             left_value = self._evaluate(node.left, environment)
144             right_value = self._evaluate(node.right, environment)

```

```

143         return self._evaluate_binary(node, left_value, right_value)
144
145     if isinstance(node, FunctionCall):
146         return self._call_function(node, environment)
147
148     raise InterpreterError(self._error_at(node, f"Unsupported
149         expression node: {type(node).__name__}"))
150
151 def _call_function(self, node: FunctionCall, environment:
152     _Environment) -> object:
153     function_runtime = self._functions.get(node.name)
154     if function_runtime is None:
155         raise InterpreterError(self._error_at(node, f"Undefined
156             function: {node.name}"))
157
158     if len(node.args) != len(function_runtime.node.params):
159         raise InterpreterError(
160             self._error_at(
161                 node,
162                 f"Function '{node.name}' expects {len(
163                     function_runtime.node.params)} argument(s), got
164                     {len(node.args)}",
165             )
166         )
167
168     call_environment = _Environment(parent=environment)
169     argument_values = [self._evaluate(argument, environment) for
170         argument in node.args]
171     for parameter_name, argument_value in zip(function_runtime.node
172         .params, argument_values, strict=False):
173         call_environment.define(parameter_name, argument_value)
174
175     try:
176         self._execute_block(function_runtime.node.body,
177             call_environment)
178     except _ReturnSignal as signal:
179         return signal.value
180     return None
181
182 def _evaluate_binary(self, node: BinaryOp, left_value: object,
183     right_value: object) -> object:
184     if node.op == "+":
185         return left_value + right_value
186     if node.op == "-":
187         return left_value - right_value
188     if node.op == "*":
189         return left_value * right_value
190     if node.op == "/":
191         return int(left_value) // int(right_value)
192     if node.op == "+.":
193         return float(left_value) + float(right_value)
194     if node.op == "-.":
195         return float(left_value) - float(right_value)

```

```

187         if node.op == "*":
188             return float(left_value) * float(right_value)
189         if node.op == "/":
190             return float(left_value) / float(right_value)
191         if node.op == "==":
192             return left_value == right_value
193         if node.op == "!=":
194             return left_value != right_value
195         raise InterpreterError(self._error_at(node, f"Unsupported
            operator: {node.op}"))
196
197     @staticmethod
198     def _format_value(value: object) -> str:
199         if isinstance(value, str):
200             return f'"{value}"'
201         if isinstance(value, bool):
202             return "true" if value else "false"
203         return str(value)
204
205     @staticmethod
206     def _runtime_type(value: object) -> LanguageType:
207         if isinstance(value, bool):
208             return LanguageType.BOOLEAN
209         if isinstance(value, int):
210             return LanguageType.INTEGER
211         if isinstance(value, float):
212             return LanguageType.FLOAT
213         if isinstance(value, str):
214             return LanguageType.STRING
215         return LanguageType.VOID
216
217     @staticmethod
218     def _error_at(node: ASTNode, message: str) -> str:
219         return f"[line {node.line}, col {node.column}] RuntimeError: {
            message}"

```

Listing 8.9: components/interpreter.py — tree-walking interpreter

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from components.ast_printer import ASTPrinter
6 from components.interpreter import Interpreter
7 from components.lexica import Lexer
8 from components.parser import Parser
9 from components.symbol_table import SymbolTable
10 from components.tokens import Token
11 from components.type_checker import TypeChecker
12
13
14 @dataclass
15 class PipelineResult:

```

```

16     tokens: list[Token]
17     ast_text: str
18     checked_scope: SymbolTable
19     outputs: list[str]
20
21
22 def run_pipeline(source_code: str) -> PipelineResult:
23     tokens = Lexer(source_code).tokenize()
24     tree = Parser(tokens).parse()
25     ast_text = ASTPrinter().print(tree)
26     checked_scope = TypeChecker().check(tree)
27
28     outputs: list[str] = []
29     Interpreter(output_callback=outputs.append).execute(tree)
30
31     return PipelineResult(
32         tokens=tokens,
33         ast_text=ast_text,
34         checked_scope=checked_scope,
35         outputs=outputs,
36     )
37
38
39 def format_tokens(tokens: list[Token]) -> str:
40     lines = []
41     for token in tokens:
42         lines.append(
43             f"    {token.token_type.name:<14} {token.lexeme!r:<12} line={
44                 token.line} col={token.column}"
45         )
46     return "\n".join(lines)
47
48 def format_stage_output(result: PipelineResult) -> str:
49     sections = [
50         "=" * 60,
51         "    STAGE 1 -- TOKENS",
52         "=" * 60,
53         format_tokens(result.tokens),
54         "",
55         "=" * 60,
56         "    STAGE 2 -- AST (parser output)",
57         "=" * 60,
58         result.ast_text,
59         "",
60         "=" * 60,
61         "    STAGE 3 -- TYPE CHECK",
62         "=" * 60,
63         result.checked_scope.format_table(),
64         "",
65         "=" * 60,
66         "    STAGE 4 -- EXECUTION",
67         "=" * 60,

```

```

68         "\n".join(result.outputs) if result.outputs else "(no output)",
69     ]
70     return "\n".join(sections)

```

Listing 8.10: components/pipeline.py — shared pipeline (CLI, UI, and tests)

A.3 Test Suite

```

1  from __future__ import annotations
2
3  import unittest
4
5  from components.lexica import Lexer, LexerError
6  from components.parser import Parser, ParseError
7  from components.pipeline import run_pipeline
8  from components.symbol_table import LanguageType, SymbolTable,
   SymbolTableError
9  from components.tokens import TokenType
10 from components.type_checker import TypeCheckError
11
12
13 #
   -----
14 # Lexer
15 #
   -----
16
17 class LexerTests(unittest.TestCase):
18     """Tests for components/lexica.py -- lexical analysis and
19     tokenisation."""
20
21     def test_integer_literal_token(self) -> None:
22         tokens = Lexer("42").tokenize()
23         self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
24         self.assertEqual(tokens[0].lexeme, "42")
25
26     def test_float_literal_token(self) -> None:
27         tokens = Lexer("3.14").tokenize()
28         self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
29         self.assertEqual(tokens[0].lexeme, "3.14")
30
31     def test_bool_literals_produce_bool_literal_tokens(self) -> None:
32         tokens = Lexer("true false").tokenize()
33         self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
34         self.assertEqual(tokens[0].lexeme, "true")
35         self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
36         self.assertEqual(tokens[1].lexeme, "false")
37
38     def test_keywords_produce_correct_token_types(self) -> None:
39         tokens = Lexer("if else while def return print").tokenize()

```

```

39     expected = [
40         TokenType.IF, TokenType.ELSE, TokenType.WHILE,
41         TokenType.DEF, TokenType.RETURN, TokenType.PRINT,
42     ]
43     actual = [t.token_type for t in tokens if t.token_type !=
44               TokenType.EOF]
45     self.assertEqual(actual, expected)
46
47     def test_identifier_not_confused_with_keyword(self) -> None:
48         tokens = Lexer("iffy whileloop").tokenize()
49         self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
50         self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)
51
52     def test_two_character_operators(self) -> None:
53         tokens = Lexer("== !=").tokenize()
54         self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
55         self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)
56
57     def test_float_dot_operators_tokenised(self) -> None:
58         tokens = Lexer("+. -. *. /.").tokenize()
59         expected = [TokenType.PLUS_DOT, TokenType.MINUS_DOT, TokenType.STAR_DOT, TokenType.SLASH_DOT]
60         actual = [t.token_type for t in tokens if t.token_type !=
61                  TokenType.EOF]
62         self.assertEqual(actual, expected)
63
64     def test_float_literal_not_confused_with_dot_operator(self) -> None:
65         :
66         # "3.14" must produce FLOAT_LITERAL, not INT_LITERAL followed
67         # by something
68         tokens = Lexer("3.14").tokenize()
69         self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
70         self.assertEqual(tokens[0].lexeme, "3.14")
71
72     def test_source_position_tracked_across_lines(self) -> None:
73         tokens = Lexer("x\ny").tokenize()
74         self.assertEqual(tokens[0].line, 1)
75         self.assertEqual(tokens[1].line, 2)
76
77     def test_unexpected_character_raises_lexer_error(self) -> None:
78         with self.assertRaises(LexerError):
79             Lexer("@").tokenize()
80
81     def test_lone_exclamation_raises_lexer_error(self) -> None:
82         with self.assertRaises(LexerError):
83             Lexer("!5").tokenize()
84
85     def test_unterminated_string_raises_lexer_error(self) -> None:
86         with self.assertRaises(LexerError):
87             Lexer('"hello').tokenize()

```

```

86 #
87 # Symbol table
88 #
89
90 class SymbolTableTests(unittest.TestCase):
91     """Tests for components/symbol_table.py -- scoped symbol table and
92     semantic types."""
93
94     def test_define_variable_and_lookup(self) -> None:
95         table = SymbolTable()
96         table.define_variable("x", LanguageType.INTEGER, initialized=
97             True)
98         symbol = table.lookup("x")
99         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
100
101     def test_duplicate_variable_definition_raises_error(self) -> None:
102         table = SymbolTable()
103         table.define_variable("x", LanguageType.INTEGER)
104         with self.assertRaises(SymbolTableError):
105             table.define_variable("x", LanguageType.FLOAT)
106
107     def test_duplicate_function_definition_raises_error(self) -> None:
108         table = SymbolTable()
109         table.define_function("f", LanguageType.INTEGER, [])
110         with self.assertRaises(SymbolTableError):
111             table.define_function("f", LanguageType.FLOAT, [])
112
113     def test_lookup_in_parent_scope(self) -> None:
114         parent = SymbolTable()
115         parent.define_variable("x", LanguageType.INTEGER, initialized=
116             True)
117         child = parent.child_scope()
118         symbol = child.lookup("x")
119         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
120
121     def test_undefined_symbol_raises_error(self) -> None:
122         table = SymbolTable()
123         with self.assertRaises(SymbolTableError):
124             table.lookup("z")
125
126     def test_format_table_contains_variable_row(self) -> None:
127         table = SymbolTable()
128         table.define_variable("counter", LanguageType.INTEGER,
129             initialized=True)
130         output = table.format_table()
131         self.assertIn("counter", output)
132         self.assertIn("variable", output)
133         self.assertIn("Integer", output)

```

```

131     def test_format_table_contains_function_row(self) -> None:
132         table = SymbolTable()
133         table.define_function("add", LanguageType.INTEGER, [("a",
134             LanguageType.INTEGER)])
135         output = table.format_table()
136         self.assertIn("add", output)
137         self.assertIn("function", output)
138
139     #
140     -----
141
142     # Parser
143     #
144     -----
145
146     class ParserTests(unittest.TestCase):
147         """Tests for components/parser.py -- recursive-descent parsing."""
148
149         def _parse(self, source: str):
150             return Parser(Lexer(source).tokenize()).parse()
151
152         def test_missing_semicolon_raises_parse_error(self) -> None:
153             with self.assertRaises(ParseError):
154                 self._parse("x = 5")
155
156         def test_unclosed_brace_raises_parse_error(self) -> None:
157             with self.assertRaises(ParseError):
158                 self._parse("if (true) { x = 1;}")
159
160         def test_operator_precedence_mul_before_add(self) -> None:
161             # 2 + 3 * 4 must be 14, not 20
162             result = run_pipeline("x = 2 + 3 * 4; print(x);")
163             self.assertEqual(result.outputs, ["14 : Integer"])
164
165         def test_operator_left_associativity(self) -> None:
166             # 10 - 3 - 2 must be 5 (left-assoc), not 9
167             result = run_pipeline("x = 10 - 3 - 2; print(x);")
168             self.assertEqual(result.outputs, ["5 : Integer"])
169
170         def test_parentheses_override_precedence(self) -> None:
171             # (2 + 3) * 4 must be 20
172             result = run_pipeline("x = (2 + 3) * 4; print(x);")
173             self.assertEqual(result.outputs, ["20 : Integer"])
174
175     #
176     -----
177
178     # Type checker
179     #
180     -----

```

```

176
177 class TypeCheckerTests(unittest.TestCase):
178     """Tests for components/type_checker.py -- static type inference and
179         checking."""
180
181     def test_integer_arithmetic_stays_integer(self) -> None:
182         result = run_pipeline("x = 3 + 4; print(x);")
183         self.assertEqual(result.outputs, ["7 : Integer"])
184
185     def test_division_always_produces_integer(self) -> None:
186         # integer / integer -> Integer (no implicit float conversion)
187         result = run_pipeline("x = 10 / 2; print(x);")
188         self.assertEqual(result.outputs, ["5 : Integer"])
189
190     def test_float_dot_operators_produce_float(self) -> None:
191         result = run_pipeline("x = 1.0 +. 2.5; print(x);")
192         self.assertEqual(result.outputs, ["3.5 : Float"])
193
194     def test_mixed_int_float_addition_raises_type_error(self) -> None:
195         # Integer + Float is no longer valid; must use +.
196         with self.assertRaises(TypeError):
197             run_pipeline("x = 1 + 2.5; print(x);")
198
199     def test_string_variable_type_inferred(self) -> None:
200         result = run_pipeline('x = "hello"; print(x);')
201         self.assertEqual(result.outputs, ['"hello" : String'])
202
203     def test_boolean_literal_type_inferred(self) -> None:
204         result = run_pipeline("x = true; print(x);")
205         self.assertEqual(result.outputs, ["true : Boolean"])
206
207     def test_boolean_equality_raises_type_error(self) -> None:
208         # Boolean == Boolean is not allowed; == only accepts arithmetic
209         # operands
210         with self.assertRaises(TypeError):
211             run_pipeline('x = true; if (x == false) { print("then"); }
212                 else { print("else"); }')
213
214     def test_arithmetic_on_string_raises_type_error(self) -> None:
215         with self.assertRaises(TypeError):
216             run_pipeline('x = "a" + 1;')
217
218     def test_if_non_boolean_condition_raises_type_error(self) -> None:
219         with self.assertRaises(TypeError):
220             run_pipeline("if (1) { x = 2; }")
221
222     def test_while_non_boolean_condition_raises_type_error(self) ->
223         None:
224         with self.assertRaises(TypeError):
225             run_pipeline("x = 0; while (x) { x = x + 1; }")
226
227     def test_assignment_type_mismatch_raises_type_error(self) -> None:

```

```

224         with self.assertRaises(TypeError):
225             run_pipeline('x = 5; x = "hello";')
226
227     def test_undefined_variable_raises_type_error(self) -> None:
228         with self.assertRaises(TypeError):
229             run_pipeline("print(z);")
230
231     def test_function_wrong_arg_count_raises_type_error(self) -> None:
232         with self.assertRaises(TypeError):
233             run_pipeline("def f(a) { return a; } f(1, 2);")
234
235     def test_function_arg_type_mismatch_raises_type_error(self) -> None:
236         :
237         # first call infers a: Integer; second call with String must
238         # fail
239         with self.assertRaises(TypeError):
240             run_pipeline('def f(a) { return a; } f(1); f("hello");')
241
242     def test_uncalled_function_body_type_error_is_caught(self) -> None:
243         # type error inside body must be detected even if function is
244         # never called
245         with self.assertRaises(TypeError):
246             run_pipeline('def bad() { y = "hello" + 1; }')
247
248     def test_valid_uncalled_function_does_not_raise(self) -> None:
249         # a well-typed but uncalled function should not raise
250         result = run_pipeline("def add(a, b) { return a + b; }")
251         self.assertEqual(result.outputs, [])
252
253     #
254     -----
255     # Integration (full pipeline)
256     #
257     -----
258
259     class IntegrationTests(unittest.TestCase):
260         """End-to-end tests through all four pipeline stages."""
261
262         # --- Unary minus ---
263
264         def test_unary_minus_integer(self) -> None:
265             result = run_pipeline("x = -5; print(x);")
266             self.assertEqual(result.outputs, ["-5 : Integer"])
267
268         def test_unary_minus_float(self) -> None:
269             result = run_pipeline("y = -. 3.14; print(y);")
270             self.assertEqual(result.outputs, ["-3.14 : Float"])
271
272         def test_unary_minus_variable(self) -> None:
273             result = run_pipeline("x = 10; y = -x; print(y);")

```

```

270         self.assertEqual(result.outputs, ["-10 : Integer"])
271
272     def test_unary_minus_inside_expression(self) -> None:
273         result = run_pipeline("x = 3 + -2; print(x);")
274         self.assertEqual(result.outputs, ["1 : Integer"])
275
276     # --- Control flow ---
277
278     def test_if_then_branch_executes(self) -> None:
279         result = run_pipeline(
280             'x = 5; if (x == 5) { print("yes"); } else { print("no"); }
281             ,
282         )
283         self.assertEqual(result.outputs, ['"yes" : String'])
284
285     def test_if_else_branch_executes_when_condition_false(self) -> None:
286         result = run_pipeline(
287             'x = 0; if (x == 5) { print("yes"); } else { print("no"); }
288             ,
289         )
290         self.assertEqual(result.outputs, ['"no" : String'])
291
292     def test_while_loop_executes_repeatedly(self) -> None:
293         result = run_pipeline("x = 3; while (x != 0) { print(x); x = x
294             - 1; }")
295         self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer",
296             "1 : Integer"])
297
298     # --- Functions ---
299
300     def test_function_integer_return(self) -> None:
301         result = run_pipeline("def square(n) { return n * n; } print(
302             square(7));")
303         self.assertEqual(result.outputs, ["49 : Integer"])
304
305     def test_function_type_inference(self) -> None:
306         result = run_pipeline(
307             "def add(a, b) { return a +. b; } result = add(2.0, 3.5);
308             print(result);"
309         )
310         self.assertEqual(result.outputs, ["5.5 : Float"])
311         self.assertIn("result          variable   Float", result.
            checked_scope.format_table())
312
313     def test_function_value_parameter_passing(self) -> None:
314         # Modifying a parameter inside a function must not affect the
315         # caller's variable
316         result = run_pipeline(
317             "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x);
318             print(x); print(y);"
319         )

```

```

312         self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer
313                                "])
314
315     def test_nested_function_calls(self) -> None:
316         result = run_pipeline(
317             "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
318         )
319         self.assertEqual(result.outputs, ["6 : Integer"])
320
321     # --- print() with all four types ---
322
323     def test_print_integer(self) -> None:
324         result = run_pipeline("print(42);")
325         self.assertEqual(result.outputs, ["42 : Integer"])
326
327     def test_print_float(self) -> None:
328         result = run_pipeline("print(3.14);")
329         self.assertEqual(result.outputs, ["3.14 : Float"])
330
331     def test_print_boolean(self) -> None:
332         result = run_pipeline("print(true);")
333         self.assertEqual(result.outputs, ["true : Boolean"])
334
335     def test_print_string(self) -> None:
336         result = run_pipeline('print("plc");')
337         self.assertEqual(result.outputs, ['"plc" : String'])
338
339     # --- Reassignment ---
340
341     def test_variable_can_be_reassigned_same_type(self) -> None:
342         result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
343         self.assertEqual(result.outputs, ["3 : Integer"])
344
345     def test_scope_isolation_if_block(self) -> None:
346         # Variable defined inside an if-block must not be visible
347         outside it
348         with self.assertRaises(TypeError):
349             run_pipeline(
350                 "x = 1; if (x == 1) { inner = 99; } print(inner);"
351             )
352
353 if __name__ == "__main__":
354     unittest.main()

```

Listing 8.11: tests/test_pipeline.py — automated regression suite (54 tests)